Studio Nx

P1-03 백엔드 공통 — 함수 설명 합본 (01~13)

2026-10-12

목차

1. ai-notes/backend(fastAPI_Python)/P1-03 - 백엔드 공통/01_errors_app_error_handler.md
2. ai-notes/backend(fastAPI_Python)/P1-03 - 백엔드 공통/02_errors_validation_error_handler.md
3. ai-notes/backend(fastAPI_Python)/P1-03 - 백엔드 공통/03_errors_http_error_handler.md
4. ai-notes/backend(fastAPI_Python)/P1-03 - 백엔드 공통/04_errors_unexpected_error_handler.md
5. ai-notes/backend(fastAPI_Python)/P1-03 - 백엔드 공통/05_security_decode_access_token.md
6. ai-notes/backend(fastAPI_Python)/P1-03 - 백엔드 공통/06_deps_get_current_user.md
7. ai-notes/backend(fastAPI_Python)/P1-03 - 백엔드 공통/07_deps_get_current_user_optional.md
8. ai-notes/backend(fastAPI_Python)/P1-03 - 백엔드 공통/08_models_user.md
9. ai-notes/backend(fastAPI_Python)/P1-03 - 백엔드 공통/09_models_inquiry.md
10. ai-notes/backend(fastAPI_Python)/P1-03 - 백엔드 공통/10_models_project.md
11. ai-notes/backend(fastAPI_Python)/P1-03 - 백엔드 공통/11_models_project_member.md
12. ai-notes/backend(fastAPI_Python)/P1-03 - 백엔드 공통/12_models_track.md
13. ai-notes/backend(fastAPI_Python)/P1-03 - 백엔드 공통/13_models_relationships.md

Backend(FastAPI/Python) 01 — `error_json()` · `app_error_handler()`

항목	내용
파일	`studionx/backend/app/core/errors.py`
작업	P1-03 백엔드 공통: 오류 응답 형식
작성	Claude (AI), 2026-10-06, 사용자 검토
다음	02 `validation_error_handler()` (입력값 형식 오류)

1. 하는 일

서버 코드가 `raise AppError(...)`로 던진 오류를 정해진 모양의 JSON 응답으로 바꿉니다.

```
raise AppError(ErrorCode.EMAIL_TAKEN, 409, "This email is already in use.")
```

위처럼 던지면 상태 코드 **409**와 다음 본문으로 응답합니다.

```
{"error": {"code": "EMAIL_TAKEN", "message": "This email is already in use.", "details": null}}
```

함수	역할
`error_json()`	오류 응답 하나를 만듭니다. 앞으로 만들 처리기 02~04도 모두 이 함수를 씁니다
`app_error_handler()`	`AppError`가 생기면 FastAPI가 부르는 함수입니다. `error_json()`으로 응답을 만듭니다

2. 언제 쓰나

서버가 "요청은 받았지만 처리할 수 없다"고 알릴 때마다 쓰입니다. 라우터와 서비스 코드는 `raise AppError(...)` 한 줄만 쓰고, 응답의 모양은 이 함수가 만듭니다.

상황 (구현 작업)	던지는 오류	화면에서 일어나는 일
가입할 때 이미 쓰이고 있는 이메일 (P1-04)	`EMAIL_TAKEN`, 409	가입 화면의 이메일 칸 아래에 "This email is already in use."가 나옵니다
로그인하지 않고 마이페이지 API 호출 (P1-03 `get_current_user`)	`UNAUTHORIZED`, 401	프론트의 `apiFetch`가 토큰을 재발급한 뒤 다시 보냅니다 (JS 02)
이메일이나 비밀번호가 틀림 (P1-05)	`INVALID_CREDENTIALS`, 401	로그인 모달에 오류 문구가 나옵니다
다른 사람의 프로젝트를 열려고 함 (P1-13)	`FORBIDDEN`, 403	접근 거부 문구가 나옵니다

FastAPI가 오류를 응답으로 바꾸는 순서

순서	일어나는 일
① 서버 시작	`main.py`가 `register_exception_handlers(app)`를 부르고, 이 함수가 "`AppError`는 `app_error_handler`가 처리한다"고 등록합니다
② 요청 처리 중	라우터나 서비스 어디에서든 `raise AppError(...)`를 던집니다

순서	일어나는 일
③ FastAPI가 오류를 잡음	오류의 종류(`AppError`)에 맞게 등록된 처리기를 찾습니다
④ 처리기 실행	`app_error_handler(request, exc)`가 JSON 응답을 만듭니다
⑤ 브라우저	`api.js`가 `error.code`를 읽어 `ApiError`로 만듭니다 (JS 02)

3. 코드

import (파일 맨 위)

```
from enum import StrEnum

from fastapi import FastAPI, Request
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse
from starlette.exceptions import HTTPException as StarletteHTTPException

from app.schemas.common import ErrorBody, ErrorResponse
```

`errors.py` 맨 위의 import입니다 (02~04에서 쓰는 것까지 모두 들어 있습니다). 이 함수들은 **`FastAPI`, `Request`, `JSONResponse`, `ErrorBody`, `ErrorResponse`**를 사용합니다. `AppError`와 `ErrorCode`는 같은 파일 안에 있습니다.

import	가져오는 것	쓰는 곳
`StrEnum` (파이썬 기본)	값이 문자열인 열거형	`ErrorCode`
`FastAPI`, `Request`	앱 타입과 요청 타입	`register_exception_handlers(app)`, 처리기의 `request`
`RequestValidationError`	입력 형식 오류의 종류	02
`JSONResponse`	JSON 응답	`error_json()`
`StarletteHTTPException`	404, 405 같은 기본 오류의 종류 (Starlette의 `HTTPException`)	03
`ErrorBody`, `ErrorResponse`	오류 응답의 모양 (Pydantic, `app/schemas/common.py`)	`error_json()`

함수

```
def error_json(
    status_code: int,
    code: str,
    message: str,
    details: dict | None = None,
) -> JSONResponse:
    body = ErrorResponse(error=ErrorBody(code=code, message=message, details=details))
    return JSONResponse(status_code=status_code, content=body.model_dump())


async def app_error_handler(request: Request, exc: AppError) -> JSONResponse:
    return error_json(exc.status_code, exc.code, exc.message, exc.details)
```

```
def register_exception_handlers(app: FastAPI) -> None:
    app.add_exception_handler(AppError, app_error_handler)
    # TODO: RequestValidationError · StarletteHTTPException · Exception 처리기 등록
```

문법 풀이

import 문법	뜻
`from 모듈 import 이름`	모듈 안의 `이름`만 가져와서 그대로 씁니다 (예: `Request`)
`app.…`	이 프로젝트(`studionx/backend/app/`)의 파일입니다. `app.core.errors`는 `app/core/errors.py`입니다

식	뜻
`status_code: int`	**타입 힌트**로, 이 값이 정수라는 표시입니다. 실행에는 영향이 없고, 편집기와 검사 도구가 씁니다
`dict | None = None`	사전(dict) 또는 `None`이며, 넘기지 않으면 `None`입니다
`-> JSONResponse`	이 함수가 돌려주는 값의 종류입니다
`ErrorResponse(error=ErrorBody(...))`	`schemas/common.py`의 Pydantic 모델입니다. 정해 둔 칸(`code`, `message`, `details`)만 받습니다
`body.model_dump()`	Pydantic 모델을 파이썬 사전으로 바꿉니다. `JSONResponse`가 이 사전을 JSON 문자열로 바꿉니다
`async def`	비동기 함수입니다. FastAPI의 오류 처리기는 이 형태로 만듭니다
`request: Request`	오류가 난 요청입니다. 지금은 쓰지 않지만 FastAPI가 항상 넘겨줍니다 (P1-22에서 API 요청과 페이지 요청을 나눌 때 씁니다)
`app.add_exception_handler(AppError, app_error_handler)`	"`AppError`가 생기면 이 함수를 불러라"라고 등록합니다

`raise AppError(...)`	`status_code`	본문 `error`
`(EMAIL_TAKEN, 409, "This email is already in use.")`	409	`{"code": "EMAIL_TAKEN", "message": "This email is already in use.", "details": null}`
`(VALIDATION_ERROR, 422, "Check the password requirements.", details={"password": ["upper", "special"]})`	422	`details`에 `{"password": ["upper", "special"]}`가 담깁니다

4. 흐름

1. 서버가 시작될 때 `AppError`를 `app_error_handler`에 연결해서 등록합니다.
2. 요청을 처리하다가 `AppError`가 생기면 FastAPI가 `app_error_handler`를 부릅니다.
3. `error_json()`이 오류에 적힌 상태 코드로 `{"error": {...}}` 응답을 만듭니다.

5. 왜 이렇게

선택	이유
응답을 만드는 함수(`error_json`)를 따로 둠	처리기가 이 함수와 02~04를 합쳐 4개가 됩니다. 응답 모양을 한 곳에서만 만들어야 서로 어긋나지 않습니다
Pydantic 모델(`ErrorResponse`)을 거쳐서 만듦	칸 이름을 잘못 쓰면 바로 오류가 납니다. 또 API 문서(`/docs`)에 나오는 모양과 실제 응답이 같아집니다
처리기는 `AppError`의 값을 그대로 씀	상태 코드와 문구는 오류를 던지는 곳이 가장 잘 압니다. 처리기는 모양만 맞춥니다
`@app.exception_handler` 대신 `add_exception_handler`	처리기를 보통 함수로 두면 시험과 문서에서 이름으로 가리킬 수 있습니다. 동작은 같습니다
`message`는 영어로 씀	화면이 영어이기 때문입니다. 화면 번역은 프론트가 `code`를 보고 합니다

6. 시험 (`tests/test_errors.py`)

시험용 작은 FastAPI 앱에 처리기만 등록하고, 일부러 `AppError`를 던지는 주소 3개로 확인했습니다.

경우	결과
`EMAIL_TAKEN`, 409	상태 409, `application/json`이고, 본문이 정확히 `{"error": {code, message, details: null}}`입니다
`details`가 있는 오류 (422)	`details`가 그대로 전달됩니다
`UNAUTHORIZED`, 401	`code`가 `"UNAUTHORIZED"`입니다. 프론트가 재발급을 시도하는 조건과 같습니다

전체 시험 17개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 02 — `validation_error_handler()`

항목	내용
파일	`studionx/backend/app/core/errors.py`
작업	P1-03 백엔드 공통: 오류 응답 형식
작성	Claude (AI), 2026-10-06, 사용자 검토
이전 · 다음	01 `error_json()` · `app_error_handler()` · 03 `http_error_handler()` (404 등 기본 오류)

1. 하는 일

요청의 입력값이 API가 정한 형식과 맞지 않을 때 FastAPI가 내는 오류를, 칸별 문구가 담긴 422 응답으로 바꿉니다.

이 함수는 유효성 검사를 하지 않습니다. 검사는 이미 끝났고, 이 함수는 검사에서 떨어진 결과를 정해진 응답 모양으로 바꿀 뿐입니다. 더 정확하게 부르자면 "형식 검사 실패 응답 변환기"입니다.

단계	하는 곳	하는 일
① 규칙 정하기	스키마 `schemas/auth.py`의 `SignupRequest`	"email은 문자열이고 255자 이하", "nickname은 꼭 있어야 함" 같은 규칙을 적어 둡니다
② 검사하기	FastAPI와 Pydantic (자동)	요청이 들어오면 ①의 규칙과 비교합니다. 틀리면 `RequestValidationError`를 만들고 라우터를 실행하지 않습니다
③ 응답 만들기	**이 함수**	②의 오류 목록을 `{"error": {"code": "VALIDATION_ERROR", "details": {칸: 문구}}}`로 바꿉니다

예를 들어 "비밀번호 8자 이상" 규칙은 이 함수가 아니라 스키마의 `Field(min_length=8)`에 있습니다. 이 함수를 지워도 검사는 그대로 일어나고, 응답만 FastAPI 기본 모양(`{"detail": [...]}`)이 됩니다.

유효성 검사는 세 곳에서 합니다

검사	하는 곳	예	오류를 내는 방법
형식 검사	스키마 (FastAPI가 자동으로)	칸 빠짐, 타입이나 길이가 틀림	`RequestValidationError`가 나면 **이 함수**가 422로 응답합니다
규칙 검사	서비스 코드 (P1-04)	비밀번호에 대문자와 기호가 있는지 (`check_password_rules`)	`raise AppError(VALIDATION_ERROR, 422, …)` → 01 처리기
DB 확인	서비스 코드 (P1-04)	이미 쓰이고 있는 이메일이나 닉네임	`raise AppError(EMAIL_TAKEN, 409, …)` → 01 처리기

화면(`join.js`, P1-07)도 입력하는 동안 같은 규칙을 검사하지만, 이것은 사용자 편의를 위한 것입니다. 화면 검사는 우회할 수 있으므로, 실제로 막는 것은 서버 검사입니다.

응답 예

가입 요청에서 `nickname`이 빠지고 비밀번호가 짧으면 다음과 같이 응답합니다.

```
422  {"error": {"code": "VALIDATION_ERROR",
                "message": "Check the highlighted fields.",
                "details": {"password": "String should have at least 8 characters",
                            "nickname": "Field required"}}}
```

2. 언제 쓰나

라우터 코드가 실행되기 **전에** 생기는 오류입니다. 라우터는 이 오류를 직접 던지지 않습니다.

순서	일어나는 일
① 요청이 들어옴	예: `POST /api/v1/auth/signup` 본문 `{"email": "a@b.com", "password": "Pw!1"}`
② FastAPI가 입력을 스키마와 비교	라우터가 받기로 한 `SignupRequest`(`schemas/auth.py`)의 칸 이름, 타입, 길이와 비교합니다
③ 맞지 않으면	`RequestValidationError`를 만들고 **라우터를 실행하지 않습니다**
④ 이 처리기 실행	422와 칸별 문구로 응답합니다
⑤ 화면	`ApiError.details`로 받아서 어느 칸이 틀렸는지 알 수 있습니다

상황	`details`
필요한 칸이 빠짐	`{"nickname": "Field required"}`
길이나 형식이 틀림	`{"password": "String should have at least 8 characters"}`
주소의 값이 숫자가 아님 (`/projects?page=abc`)	`{"page": "Input should be a valid integer, …"}`
본문 자체가 없음	`{"body": "Field required"}`

화면 문구는 프론트가 정합니다. 가입 화면은 입력할 때 이미 규칙을 검사하고 자기 문구를 보여 줍니다(P1-07). 이 422는 화면 검사를 거치지 않고 API를 직접 부른 경우를 막는 마지막 단계이며, `details`의 영어 문구는 개발과 디버깅용입니다.

3. 코드

import (파일 맨 위)

```
from enum import StrEnum

from fastapi import FastAPI, Request
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse
from starlette.exceptions import HTTPException as StarletteHTTPException

from app.schemas.common import ErrorBody, ErrorResponse
```

`errors.py` 맨 위의 import입니다. 이 함수는 **`Request`, `RequestValidationError`, `JSONResponse`**를 사용합니다. `error_json`과 `ErrorCode`는 같은 파일 안에 있습니다.

함수

```
async def validation_error_handler(request: Request, exc: RequestValidationError) -> JSONResponse:
    details: dict[str, str] = {}
    for err in exc.errors():
```

```
        loc = err.get("loc") or ()
        field = str(loc[-1]) if loc else "body"
        details.setdefault(field, err.get("msg", "Invalid value."))
    return error_json(422, ErrorCode.VALIDATION_ERROR, "Check the highlighted fields.", details)
```

`register_exception_handlers()`에 한 줄을 추가했습니다.

```
app.add_exception_handler(RequestValidationError, validation_error_handler)
```

문법 풀이

import 문법	뜻
`from 모듈 import 이름`	모듈 안의 `이름`만 가져와서 그대로 씁니다 (예: `Request`)

식	뜻
`exc.errors()`	오류 목록입니다. 칸마다 사전이 하나씩 들어 있습니다: `{"loc": ("body", "nickname"), "msg": "Field required", "input": …, …}`
`err.get("loc") or ()`	`loc`가 없거나 비어 있으면 빈 튜플 `()`을 씁니다
`loc[-1]`	마지막 값입니다. `("body", "nickname")`이면 `"nickname"`입니다
`str(...)`	목록 칸의 오류는 위치가 숫자(`("body", "tags", 0)`의 `0`)라서 문자열로 바꿉니다
`A if 조건 else B`	조건이 참이면 A, 아니면 B입니다 (JS의 `조건 ? A : B`)
`details.setdefault(field, msg)`	`field`가 아직 없을 때만 넣습니다. 그래서 **칸마다 첫 번째 문구만** 남습니다

loc	칸 이름
`("body", "email")`	`email`
`("query", "page")`	`page`
`("body",)` (본문 없음)	`body`
`("body", "tags", 0)`	`0`

4. 흐름

1. FastAPI가 입력 오류 목록을 넘겨줍니다.
2. 오류마다 `loc`의 마지막 값을 칸 이름으로, `msg`를 문구로 꺼냅니다.
3. 같은 칸에 대해서는 첫 번째 문구만 남깁니다.
4. `error_json()`으로 422 응답을 만듭니다 (01).

5. 왜 이렇게

선택	이유
FastAPI 기본 응답(`{"detail": [...]}`)을 바꿈	기본 모양은 다른 오류와 달라서 `api.js`가 `error.code`를 읽을 수 없습니다. 모든 오류를 `{"error": {...}}` 하나로 맞춥니다
보낸 값(`input`)은 응답에 넣지 않음	기본 응답은 사용자가 보낸 값을 그대로 돌려줍니다. 그러면 비밀번호 칸이 틀렸을 때 비밀번호가 응답과 로그에 남습니다 (PRD 01 §5). 그래서 칸 이름과 문구만 보냅니다

선택	이유
칸마다 문구 하나	화면은 칸 아래에 한 줄만 보여 줍니다. 사전 `{칸: 문구}` 모양이면 프론트가 `details.email`로 바로 꺼낼 수 있습니다
칸 이름은 `loc`의 마지막 값	화면의 입력 칸 이름(`email`, `nickname`)과 같아집니다. 앞부분(`body`, `query`)은 화면에 필요하지 않습니다
전체 문구는 하나로 고정	칸별 내용은 `details`에 있으므로, `message`는 "표시된 칸을 확인하라"는 안내만 합니다

6. 시험 (`tests/test_errors.py`에 3개 추가)

시험용 앱에 가입과 같은 모양의 스키마(`email`, 8자 이상의 `password`, `nickname`)와 `page: int`를 받는 주소를 만들어서 확인했습니다.

경우	결과
이메일 형식이 틀리고 `nickname`이 빠짐	422 `VALIDATION_ERROR`이고, `details`의 칸이 `email`과 `nickname` 두 개이며, `nickname`은 `"Field required"`입니다
짧은 비밀번호 `Pw!1`	`details`에 `password`가 있고, 응답 어디에도 `Pw!1`이 없습니다
`/search?page=abc`	`details`에 `page`가 있습니다
본문 없이 POST	422 `VALIDATION_ERROR`

전체 시험 20개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 03 — `http_error_handler()`

항목	내용
파일	`studionx/backend/app/core/errors.py`
작업	P1-03 백엔드 공통: 오류 응답 형식
작성	Claude (AI), 2026-10-06, 사용자 검토
이전 · 다음	02 `validation_error_handler()` · 04 `unexpected_error_handler()` (예상하지 못한 오류 500)

1. 하는 일

이 프로젝트의 코드가 아니라 **FastAPI와 Starlette가 직접 내는 오류**를 다른 오류와 같은 모양으로 바꿉니다.

`GET /api/v1/no-such-endpoint` (없는 API 주소)의 경우:

	응답
바꾸기 전 (FastAPI 기본)	`404 {"detail": "Not Found"}`
바꾼 뒤	`404 {"error": {"code": "NOT_FOUND", "message": "Not found.", "details": null}}`

2. 언제 쓰나

라우터 코드가 실행되지 않았거나, 라이브러리가 스스로 요청을 거절할 때 생깁니다.

상황	상태	code
없는 API 주소 (`/api/v1/projectz`)	404	`NOT_FOUND`
주소는 있는데 방식이 다름 (POST만 받는 `/auth/login`에 GET)	405	`METHOD_NOT_ALLOWED`
라이브러리가 요청 자체를 거절 (깨진 요청 등)	400	`BAD_REQUEST`

이 프로젝트의 코드가 일부러 내는 오류는 `AppError`로 던지고 01이 처리합니다. 이 함수는 그 밖의 기본 오류를 맡습니다.

오류 처리기 네 개의 역할

#	처리기	맡는 오류	오류를 내는 쪽
01	`app_error_handler`	`AppError`	이 프로젝트의 코드 (일부러 냄)
02	`validation_error_handler`	입력 형식 오류	FastAPI (스키마와 비교)
03	**`http_error_handler`**	404, 405 등	FastAPI와 Starlette (주소와 방식)
04	`unexpected_error_handler`	그 밖의 모든 오류	버그, DB 연결 끊김 등 (예상하지 못한 오류)

페이지 주소는 다르게 처리합니다

없는 **페이지** 주소(`/no-such-page`)는 이 함수까지 오지 않습니다. `core/site_files.py`가 먼저 404 오류 페이지(HTML)를 돌려줍니다. API 주소(`/api/...`)만 이 함수로 옵니다.

주소	응답
`/no-such-page`	404 오류 페이지 HTML ("Lost the signal")
`/api/v1/no-such-endpoint`	404 JSON (이 함수)

메시지가 보이는 곳

JSON 응답은 사람이 아니라 **페이지의 JS(`api.js`)가 받습니다.** 화면에 문구를 보여 줄지는 그 페이지의 JS가 정합니다.

단계	일어나는 일
① 서버	`{"error": {"code": "NOT_FOUND", "message": "Not found."}}`로 응답합니다
② `api.js`	`ApiError`로 바꿉니다. `err.code`는 `"NOT_FOUND"`, `err.message`는 `"Not found."`입니다 (JS 02)
③ 페이지 JS	`catch (err)`로 받아서 화면에 표시합니다 (칸 아래 문구, 알림, 모달 안 문구). `code`를 보고 문구를 직접 정할 수도 있습니다

페이지 JS(P1-07 이후)가 아직 없어서 지금은 화면에서 볼 수 없습니다. 다음 방법으로 확인합니다.

확인 방법	위치
브라우저 개발자 도구	`F12` → Network 탭 → 요청 → Response
API 문서 화면	`http://localhost:8000/docs` → API 선택 → Try it out
주소창에 API 주소 입력	`localhost:8000/api/v1/abc`를 입력하면 JSON 문자열이 그대로 보입니다

오류 HTML 페이지(`frontend/errors/*.html`)와의 관계

요청은 두 종류이고, 오류도 요청의 종류에 따라 다르게 돌려줍니다.

	페이지 요청	API 요청
보내는 쪽	**사람**: 주소창 입력, 링크 클릭	**JS**: `api.get(...)`
주소	`/myPage.html`, `/abc`	`/api/v1/...`
오류 응답	**오류 HTML 페이지** (404, 500 등을 디자인한 화면)	**JSON** (01~04 처리기)
사용자가 보는 것	오류 페이지 전체 화면	지금 보던 화면 안의 문구

API 오류에 HTML을 주면 안 됩니다. 가입 버튼을 눌렀는데 이메일이 중복이라는 이유로 화면 전체가 오류 페이지로 바뀌면 안 되기 때문입니다. API는 JSON으로 답하고, 화면은 그대로 둔 채 문구만 보여 줍니다.

경우	지금	할 작업
없는 페이지 `/abc`	✅ `errors/404.html` (`site_files.py`)	없음
없는 API `/api/v1/abc`	✅ 404 JSON (이 처리기)	없음
페이지 요청 중 서버 오류 (500, 403 등)	❌ JSON이 나갑니다	**P1-22**: 처리기에서 주소가 `/api`로 시작하지 않으면 `errors/500.html` 등을 보냅니다

`errors.py`의 `ERROR_PAGES` 표(`500 → errors/500.html` …)가 P1-22에서 쓸 연결 고리입니다. 01~04 처리기에 "API 요청이면 JSON, 페이지 요청이면 HTML"이라는 갈림길만 더하면 됩니다.

API 오류가 나면 JS가 오류 페이지로 보내는 경우 (2026-10-06 결정)

페이지를 열 때 부르는 **본 데이터**를 받지 못하면, 그 페이지는 보여 줄 것이 없습니다. 이때는 JS가 오류 페이지로 이동합니다. 버튼이나 폼 동작의 오류는 화면 안 문구로 보여 줍니다.

오류가 난 요청	예	처리
본 데이터: 404	`studioNx.html?projectId=999` → `GET /projects/999/full`	`location.replace('errors/404.html')`
본 데이터: 403	다른 사람의 프로젝트를 주소로 열기	`location.replace('errors/403.html')`
본 데이터: 500 이상	대시보드나 프로젝트를 불러오다가 서버 오류	`location.replace('errors/500.html')`
본 데이터: 401	로그인이 풀림	오류 페이지가 아니라 `requireLogin()` 흐름 (로그인 모달)
버튼과 폼 동작 (저장, 가입, 이름 변경 ...)	이메일 중복, 이름 변경 실패	화면 안 문구로 보여 주고, 페이지는 그대로 둡니다

`replace`로 이동하는 이유는 JS 07과 같습니다. 뒤로 가기로 오류가 난 페이지에 다시 들어가지 않게 하기 위해서입니다.

적용할 곳 (각 함수의 TODO 주석에 적어 두었습니다)	본 데이터
시퀀서 `sequencer.js`의 `loadProject()`	`GET /projects/{id}/full`
마이페이지 `mypage.js`의 `loadDashboard()` (P1-17)	`GET /me/dashboard`
회원 정보 `userinfo.js`의 `loadMe()` (P1-10)	`GET /users/me`
내 프로젝트 `myproject.js`의 `loadProjects()` (P1-14)	`GET /projects`: 목록이라서 404와 403은 없고 500 이상만 처리합니다

3. 코드

import (파일 맨 위)

```
from enum import StrEnum

from fastapi import FastAPI, Request
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse
from starlette.exceptions import HTTPException as StarletteHTTPException

from app.schemas.common import ErrorBody, ErrorResponse
```

`errors.py` 맨 위의 import입니다. 이 함수는 **`Request`, `JSONResponse`, `StarletteHTTPException`**을 사용합니다.

함수

```
HTTP_ERRORS: dict[int, tuple[ErrorCode, str]] = {
    400: (ErrorCode.BAD_REQUEST, "Bad request."),
    401: (ErrorCode.UNAUTHORIZED, "Please sign in."),
    403: (ErrorCode.FORBIDDEN, "You do not have permission."),
    404: (ErrorCode.NOT_FOUND, "Not found."),
    405: (ErrorCode.METHOD_NOT_ALLOWED, "Method not allowed."),
    413: (ErrorCode.FILE_TOO_LARGE, "The file is too large."),
```

```
    415: (ErrorCode.UNSUPPORTED_FILE_TYPE, "Unsupported file type."),
    429: (ErrorCode.TOO_MANY_REQUESTS, "Too many requests. Please try again later."),
}


async def http_error_handler(request: Request, exc: StarletteHTTPException) -> JSONResponse:
    if exc.status_code in HTTP_ERRORS:
        code, message = HTTP_ERRORS[exc.status_code]
    elif exc.status_code >= 500:
        code, message = ErrorCode.INTERNAL_ERROR, "Something went wrong. Please try again."
    else:
        code, message = ErrorCode.BAD_REQUEST, "Bad request."
    response = error_json(exc.status_code, code, message)
    if exc.headers:
        response.headers.update(exc.headers)
    return response
```

`register_exception_handlers()`에 등록하는 줄 하나를 더하고, `ErrorCode`에 `BAD_REQUEST`와 `METHOD_NOT_ALLOWED` 두 개를 추가했습니다.

문법 풀이

import 문법	뜻
`from 모듈 import 이름`	모듈 안의 `이름`만 가져와서 그대로 씁니다 (예: `Request`)
`import A as B`	가져오면서 이름을 `B`로 바꿉니다
`HTTPException as StarletteHTTPException`	FastAPI에도 `HTTPException`이 있어서, Starlette 쪽이라는 것을 이름으로 드러냅니다

식	뜻
`dict[int, tuple[ErrorCode, str]]`	키는 정수(상태 코드)이고, 값은 `(code, 문구)` 두 개로 된 묶음입니다
`exc.status_code in HTTP_ERRORS`	사전에 그 키가 있는지 확인합니다
`code, message = HTTP_ERRORS[...]`	두 개 묶음을 두 변수에 나눠 담습니다 (**언패킹**)
`if … elif … else`	순서대로 확인합니다. `elif`는 JS의 `else if`입니다
`response.headers.update(exc.headers)`	오류에 딸린 헤더를 응답에 덧붙입니다
`StarletteHTTPException`	FastAPI가 쓰는 Starlette의 `HTTPException`입니다. FastAPI의 `HTTPException`은 이것을 물려받으므로, 둘 다 이 처리기로 옵니다

상태	`code`	`message`
404	`NOT_FOUND`	`Not found.`
405	`METHOD_NOT_ALLOWED`	`Method not allowed.`
418 (표에 없고 500 미만)	`BAD_REQUEST`	`Bad request.`
502 (표에 없고 500 이상)	`INTERNAL_ERROR`	`Something went wrong. Please try again.`

4. 흐름

1. 상태 코드가 표(`HTTP_ERRORS`)에 있으면 표의 `code`와 문구를 씁니다.
2. 표에 없으면 500 이상은 `INTERNAL_ERROR`, 그 밖은 `BAD_REQUEST`로 정합니다.
3. `error_json()`으로 같은 상태 코드의 응답을 만듭니다 (01).
4. 오류에 헤더가 있으면 응답에 붙입니다.

5. 왜 이렇게

선택	이유
모양을 `{"error": {...}}`로 맞춤	`api.js`는 `error.code`만 읽습니다. 기본 모양인 `{"detail": …}`으로 두면 모든 404가 `INTERNAL_ERROR`로 보입니다 (JS 02 `toApiError`)
문구는 표에 정해 둔 고정 문장	`exc.detail`은 라이브러리나 다른 코드가 만든 문장이라서 내부 정보(서버 주소 등)가 섞일 수 있습니다. 응답에는 정해 둔 문장만 내보냅니다
상태 코드는 그대로 둠	404는 404로 둡니다. 상태를 바꾸면 브라우저, 검색 엔진, 모니터링이 잘못 판단합니다
헤더를 유지함	405 응답에는 받을 수 있는 방식을 알려 주는 `Allow` 헤더가 표준으로 필요합니다
`ErrorCode`에 두 개 추가	405와 그 밖의 400을 나타낼 코드가 없었습니다. 프론트가 분기할 일은 거의 없지만, `code`가 비어 있으면 안 됩니다
401, 403, 413, 415, 429도 표에 둠	지금은 이 프로젝트의 코드가 `AppError`로 내지만, 라이브러리가 같은 상태를 내더라도 같은 `code`가 나오게 합니다

6. 시험 (`tests/test_errors.py`에 4개 추가)

경우	결과
시험 앱의 없는 주소	404이고, 본문이 정확히 `{"error": {"code": "NOT_FOUND", "message": "Not found.", "details": null}}`입니다
GET만 있는 주소에 POST	405 `METHOD_NOT_ALLOWED`이고, `Allow: GET` 헤더가 유지됩니다
418과 502를 내부 문장과 함께 던짐	각각 `BAD_REQUEST`와 `INTERNAL_ERROR`이고, 내부 문장(`10.0.0.5` 등)은 응답에 없습니다
실제 앱의 `/api/v1/no-such-endpoint`와 `/no-such-page`	API는 404 JSON `NOT_FOUND`이고, 페이지는 404 HTML 그대로입니다

전체 시험 24개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 04 — `unexpected_error_handler()`

항목	내용
파일	`studionx/backend/app/core/errors.py`
작업	P1-03 백엔드 공통: 오류 응답 형식 (처리기 네 개 중 마지막)
작성	Claude (AI), 2026-10-06, 사용자 검토
이전 · 다음	03 `http_error_handler()` · 05 `decode_access_token()` (토큰 검사)

1. 하는 일

01~03이 맡지 않은 **모든 오류**(버그, DB 연결 끊김처럼 예상하지 못한 오류)를 500 응답으로 바꿉니다. 응답에는 일반적인 문장만 넣고, 오류 내용은 넣지 않습니다.

```
500  {"error": {"code": "INTERNAL_ERROR", "message": "Something went wrong. Please try again.", "detail
s": null}}
```

2. 언제 쓰나

코드가 예상하지 못한 이유로 멈췄을 때입니다. 정상이라면 일어나지 않아야 하는 오류입니다.

상황	원래 오류 (서버 안)	응답
코드 버그 (`None`의 칸을 읽는 등)	`AttributeError: 'NoneType' object has no attribute 'email'`	500, 일반 문장
DB 서버가 꺼짐	`OperationalError: (2003, "Can't connect to MySQL server on 'localhost'")`	500, 일반 문장
파일 저장 실패	`PermissionError: [Errno 13] … C:\Users\…\uploads\…`	500, 일반 문장

오류 내용이 남는 곳

응답에는 없지만 오류 내용이 **사라지지는 않습니다.** 이 처리기가 응답을 보낸 뒤 Starlette가 원래 오류를 다시 던지고, 두 곳이 그것을 기록합니다.

순서	일어나는 일
① 라우터에서 오류	`RuntimeError(...)`가 납니다
② 이 처리기	사용자에게 500과 일반 문장으로 응답합니다
③ Starlette	응답을 보낸 뒤 **원래 오류를 다시 던집니다** (`raise exc`)
④ uvicorn	서버를 실행한 터미널에 오류 전체(어느 파일의 몇째 줄인지)를 출력합니다
⑤ Sentry	`SENTRY_DSN`이 설정되어 있으면 Sentry로 보냅니다 (`core/sentry.py`가 비밀번호, 토큰, 쿠키를 지운 뒤에 보냅니다)

보는 사람	보는 것
사용자 (화면)	"Something went wrong. Please try again."

보는 사람	보는 것
개발자	터미널 로그와 Sentry에서 오류 종류, 내용, 위치 전체를 봅니다

처리기 네 개 정리 (P1-03 오류 응답 형식 완료)

#	처리기	맡는 오류	상태
01	`app_error_handler`	이 프로젝트의 코드가 일부러 던진 `AppError`	오류에 적힌 값
02	`validation_error_handler`	입력 형식 오류	422
03	`http_error_handler`	없는 주소, 다른 방식 등	404, 405 …
04	**`unexpected_error_handler`**	그 밖의 모든 오류	500

3. 코드

import (파일 맨 위)

```
from enum import StrEnum

from fastapi import FastAPI, Request
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse
from starlette.exceptions import HTTPException as StarletteHTTPException

from app.schemas.common import ErrorBody, ErrorResponse
```

`errors.py` 맨 위의 import입니다. 이 함수는 **`Request`와 `JSONResponse`**만 사용합니다. `Exception`은 파이썬 기본이라 import할 필요가 없습니다.

함수

```
async def unexpected_error_handler(request: Request, exc: Exception) -> JSONResponse:
    return error_json(500, ErrorCode.INTERNAL_ERROR, "Something went wrong. Please try again.")
```

`register_exception_handlers()`에 등록합니다.

```
app.add_exception_handler(Exception, unexpected_error_handler)
```

문법 풀이

import 문법	뜻
`from 모듈 import 이름`	모듈 안의 `이름`만 가져와서 그대로 씁니다 (예: `Request`)

식	뜻
`exc: Exception`	`Exception`은 거의 모든 오류의 부모입니다. 그래서 01~03이 맡지 않은 오류가 모두 여기로 옵니다
`exc`를 쓰지 않음	일부러 그렇게 했습니다. 오류 내용을 응답에 넣지 않으므로 받기만 합니다
`add_exception_handler(Exception, …)`	FastAPI는 `Exception` 처리기를 다른 처리기와 따로, **가장 바깥**(`ServerErrorMiddleware`)에 둡니다. 더 구체적인 처리기(01~03)가 먼저 잡고, 아무도 잡지 않은 오류만 여기로 옵니다

4. 흐름

1. 01~03이 잡지 않은 오류가 가장 바깥까지 올라옵니다.
2. 이 처리기가 500과 일반 문장으로 응답을 만듭니다.
3. Starlette가 응답을 보낸 뒤 원래 오류를 다시 던지고, 터미널 로그와 Sentry가 그것을 기록합니다.

5. 왜 이렇게

선택	이유
오류 내용을 응답에 넣지 않음	오류 문장에는 SQL, 파일 경로, DB 주소, 비밀값이 섞일 수 있습니다 (§2의 예). 그대로 보내면 공격자에게 서버 구조를 알려 주게 됩니다 (PRD 01 §5)
기록하는 코드를 따로 쓰지 않음	Starlette가 오류를 다시 던져서 uvicorn과 Sentry가 이미 기록합니다. 여기서 또 기록하면 Sentry에 같은 오류가 두 번 쌓입니다
응답 모양을 다른 오류와 같게 맞춤	`api.js`가 `INTERNAL_ERROR`로 읽을 수 있습니다. 기본 500 응답은 JSON이 아니라 `Internal Server Error`라는 글자라서, 프론트가 형식을 알 수 없습니다
문구는 "다시 시도"	DB 연결이 잠깐 끊긴 경우처럼 다시 하면 되는 일이 많습니다. 오류가 계속되면 Sentry에서 알 수 있습니다

6. 시험 (`tests/test_errors.py`에 2개 추가)

시험용 주소가 `RuntimeError("SELECT * FROM users WHERE password_hash='…'")`를 던지게 했습니다.

경우	결과
응답 확인 (`raise_server_exceptions=False`)	500이고, 본문이 정확히 위의 JSON이며, 응답에 `password_hash`와 `RuntimeError`가 없습니다
다시 던지는지 확인 (기본 설정)	시험 도구에서 `RuntimeError`가 그대로 올라옵니다. 그래서 로그와 Sentry가 기록할 수 있습니다

`raise_server_exceptions=False`: 시험 도구(`TestClient`)는 기본적으로 서버 오류를 다시 던져서 시험을 멈춥니다. 실제 응답을 보려고 이 옵션을 껐습니다.

전체 시험 26개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 05 — `decode_access_token()` · `_decode_token()`

항목	내용
파일	`studionx/backend/app/core/security.py`
작업	P1-03 백엔드 공통: 토큰 검사 (PRD 01 FR-105)
작성	Claude (AI), 2026-10-06, 사용자 검토
이전 · 다음	04 `unexpected_error_handler()` · 06 `get_current_user()`

1. 하는 일

요청에 실려 온 Access Token이 **이 서버가 만든 진짜 토큰이고 아직 유효한지** 확인한 뒤, 토큰 주인의 `user_id`를 돌려줍니다. 문제가 있으면 401 오류를 던집니다.

`decode_access_token("eyJhbGciOi…")` → `7` (회원 번호)

함수	역할
`decode_access_token()`	Access Token을 검사합니다. 바깥에서 부르는 이름입니다
`_decode_token()`	실제로 검사하는 곳입니다. P1-05의 `decode_refresh_token()`도 같은 함수를 씁니다 (`type`만 다릅니다)

2. 언제 쓰나

로그인이 필요한 **모든 API 요청마다** 한 번씩 불립니다.

순서	일어나는 일
① 브라우저	`GET /api/v1/me/dashboard`를 헤더 `Authorization: Bearer eyJhbGciOi…`와 함께 보냅니다 (JS 02)
② `get_current_user()` (다음 06번)	헤더에서 토큰 문자열을 꺼내 **이 함수**에 넘깁니다
③ 이 함수	검사한 뒤 `7`을 돌려줍니다
④ `get_current_user()`	DB에서 7번 회원을 찾아 라우터에 넘깁니다
실패하면	401 `UNAUTHORIZED`로 응답하고, 브라우저의 `api.js`가 리프레시 쿠키로 새 토큰을 받아 한 번 다시 보냅니다

JWT(토큰)의 생김새

토큰은 점(`.`)으로 나뉜 세 부분으로 되어 있습니다.

부분	예	내용
머리	`eyJhbGciOiJIUzI1NiJ9`	서명 방식 `{"alg": "HS256"}`
내용	`eyJzdWIiOiI3Ii…`	`{"sub": "7", "type": "access", "exp": 1791273600}`
서명	`kQ3x…`	머리와 내용을 **서버 비밀값(`JWT_SECRET`)**으로 계산한 값

칸	뜻
`sub`	토큰의 주인인 회원 번호입니다 (문자열로 저장합니다)
`type`	`access`(30분, API 요청용) 또는 `refresh`(3일, 재발급용)입니다
`exp`	만료 시각입니다 (1970년부터 센 초)

내용은 암호화되지 않습니다. 누구나 풀어서 읽을 수 있으므로 비밀번호 같은 것은 넣지 않습니다. 대신 **서명** 덕분에 내용을 바꿀 수 없습니다. `sub`를 `7`에서 `1`로 바꾸면 서명이 맞지 않게 되고, 비밀값을 모르는 사람은 맞는 서명을 만들 수 없습니다.

3. 코드

import (파일 맨 위)

```
import jwt

from app.core.config import get_settings
from app.core.errors import AppError, ErrorCode
```

`security.py` 맨 위의 import입니다.

import	가져오는 것	쓰는 곳
`jwt`	PyJWT 라이브러리 (`requirements.txt`의 `pyjwt`)	`jwt.decode`, `jwt.PyJWTError`
`get_settings`	`.env`를 읽어 만든 설정 (`app/core/config.py`)	`get_settings().jwt_secret`
`AppError`, `ErrorCode`	01번의 오류 클래스와 코드 (`app/core/errors.py`)	401 `UNAUTHORIZED`

설치할 때의 이름(`pyjwt`)과 import할 때의 이름(`jwt`)이 다릅니다.

함수

```
JWT_ALGORITHM = "HS256"


def decode_access_token(token: str) -> int:
    return _decode_token(token, "access")


def _decode_token(token: str, expected_type: str) -> int:
    invalid = AppError(ErrorCode.UNAUTHORIZED, 401, "Please sign in again.")
    try:
        payload = jwt.decode(
            token,
            get_settings().jwt_secret,
            algorithms=[JWT_ALGORITHM],
            options={"require": ["exp", "sub", "type"]},
        )
    except jwt.PyJWTError:
        raise invalid from None
    if payload["type"] != expected_type:
        raise invalid
    try:
        return int(payload["sub"])
```

```
    except (TypeError, ValueError):
        raise invalid from None
```

문법 풀이

import 문법	뜻
`import 모듈`	모듈 전체를 가져와서 `모듈.이름` 으로 씁니다 (예: `jwt.decode`)
`from 모듈 import 이름`	모듈 안의 `이름` 만 가져와서 그대로 씁니다 (예: `Request`)
`app.…`	이 프로젝트(`studionx/backend/app/`)의 파일입니다. `app.core.errors` 는 `app/core/errors.py` 입니다

식	뜻
`jwt.decode(token, 비밀값, algorithms=[…])`	PyJWT 라이브러리의 함수입니다. 서명과 만료(`exp`)를 확인한 뒤 내용을 사전으로 돌려줍니다. 하나라도 틀리면 오류를 냅니다
`options={"require": [...]}`	이 칸들이 없으면 실패합니다
`except jwt.PyJWTError:`	PyJWT의 모든 오류(만료 `ExpiredSignatureError`, 서명 틀림 `InvalidSignatureError`, 형식 틀림 `DecodeError` …)의 부모입니다. 한 번에 잡습니다
`raise invalid from None`	이 프로젝트의 오류로 바꿔 던집니다. `from None` 은 "원래 오류는 이어 붙이지 말라"는 뜻이라서 로그가 짧아집니다
`int(payload["sub"])`	`"7"` 을 `7` 로 바꿉니다. `"admin"` 이면 `ValueError`, 숫자가 아닌 종류면 `TypeError` 가 납니다
`_decode_token` 의 `_`	이 파일 안에서만 쓰는 함수라는 파이썬 관례입니다

토큰	결과
정상 (`sub: "7"`, `type: access`, 만료 전)	`7`
만료됨	401
다른 비밀값으로 서명함	401
내용만 바꿈 (`sub` 7 → 1)	401 (서명 불일치)
`type: refresh`	401
`sub`, `type`, `exp` 중 하나가 없음	401
`sub: "admin"`	401
서명이 없는 토큰 (`alg: none`)	401
`"not-a-token"`, `""`	401

4. 흐름

1. 서명, 만료, 필수 칸을 확인합니다 (`jwt.decode`). 실패하면 401로 응답합니다.
2. `type` 이 `access` 인지 확인합니다. 아니면 401로 응답합니다.
3. `sub` 를 정수로 바꿔서 돌려줍니다. 바꿀 수 없으면 401로 응답합니다.

5. 왜 이렇게 (보안)

선택	이유
`algorithms`를 `HS256` 하나로 고정	토큰 머리의 `alg`는 보내는 사람이 마음대로 쓸 수 있습니다. `"none"`(서명 없음)을 믿으면 누구나 `sub: 1` 토큰을 만들 수 있습니다. 그래서 서버가 정한 방식만 받습니다
`type`을 확인	리프레시 토큰(3일)을 Access Token 자리에 넣어서 30분 제한을 피하지 못하게 합니다
실패 이유와 상관없이 같은 401과 같은 문구	만료인지 위조인지 알려 주면 공격자에게 힌트가 됩니다. 프론트는 어차피 같은 처리(재발급)를 합니다
`"Please sign in again."`의 코드는 `UNAUTHORIZED`	프론트 `api.js`가 재발급을 시도하는 조건과 같은 코드입니다 (JS 02)
검사 부분을 `_decode_token()`으로 분리	P1-05의 `decode_refresh_token()`은 `return _decode_token(token, "refresh")` 한 줄이 됩니다
비밀값은 `.env`의 `JWT_SECRET`에 둠	코드나 Git에 넣지 않습니다. 32바이트 이상이어야 합니다 (짧으면 PyJWT가 경고합니다). 지금 로컬 값은 64바이트이며, 값은 출력하지 않고 길이만 확인했습니다

비밀값이 새어 나가면 누구나 아무 회원의 토큰을 만들 수 있습니다. 그때는 `JWT_SECRET`을 바꾸면 기존 토큰이 모두 무효가 됩니다 (모든 사용자가 다시 로그인해야 합니다).

6. 시험 (`tests/test_security.py` 새 파일, 9개)

토큰 발급 함수(`create_access_token`)는 P1-05에서 만들기 때문에, 시험에서는 `jwt.encode`로 토큰을 직접 만들었습니다.

경우	결과
정상 토큰	`7`
만료, 다른 비밀값, `type: refresh`	401 `UNAUTHORIZED` "Please sign in again."
`sub`, `type`, `exp` 중 하나가 없음 (3개)	401
`sub: "admin"`	401
`"not-a-token"`, 빈 문자열	401
서명이 없는 토큰 (`alg: none`)	401
내용만 바꾼 토큰	401

전체 시험 35개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 06 — `get_current_user()`

항목	내용
파일	`studionx/backend/app/api/deps.py`
작업	P1-03 백엔드 공통: 로그인한 사용자 찾기 (PRD 01 FR-105, FR-108)
작성	Claude (AI), 2026-10-06, 사용자 검토
이전 · 다음	05 `decode_access_token()` · 07 `get_current_user_optional()`

1. 하는 일

요청 헤더의 Access Token으로 **요청한 회원을 DB에서 찾아** 라우터에 넘깁니다. 토큰이 없거나, 틀렸거나, 회원이 활동 중이 아니면 401 오류를 던집니다.

`Authorization: Bearer eyJhbGciOi…` → `User(user_id=7, …)`

2. 언제 쓰나

로그인이 필요한 API가 함수 인자에 `user: CurrentUser`라고 적어 두면, FastAPI가 라우터보다 **먼저** 이 함수를 부릅니다. 그래서 라우터는 검사가 끝난 회원만 받습니다.

```
@router.get("/users/me")
def read_me(user: CurrentUser) -> MeResponse:   # user = 이 함수가 돌려준 회원
    ...
```

이 함수를 쓰는 API (지금 뼈대 기준)	화면
`GET /users/me`, `PATCH /users/me`, 프로필 사진, 비밀번호 변경, 탈퇴	userInfo.html
`GET /me/dashboard`	myPage.html
`GET`, `POST`, `PATCH`, `DELETE /projects…`	myProject.html, studioNx.html
쓰지 않는 API: `POST /inquiries`	로그인이 선택이라서 07 `get_current_user_optional()`을 씁니다

순서	일어나는 일
① 브라우저	`GET /api/v1/me/dashboard`를 `Authorization: Bearer eyJ…`와 함께 보냅니다 (JS 02)
② `bearer_scheme`	헤더에서 `eyJ…`만 꺼냅니다
③ `get_db`	DB 세션을 엽니다
④ **이 함수**	토큰을 검사하고(05) 7을 얻은 뒤, DB에서 7번 회원을 찾아 상태를 확인합니다
⑤ 라우터	`user`로 7번 회원을 받아 대시보드를 만듭니다
실패하면	라우터는 실행되지 않습니다. 401로 응답하면 브라우저의 `api.js`가 재발급을 받은 뒤 한 번 다시 보냅니다

3. 코드

import (파일 맨 위)

```
from typing import Annotated

from fastapi import Depends
from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer
from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.core.security import decode_access_token
from app.db.session import get_db
from app.models import User
```

`deps.py` 맨 위의 import입니다. **`AppError`, `ErrorCode`, `decode_access_token` 두 줄을 이번에 추가했습니다.**

import	가져오는 것	쓰는 곳
`Annotated` (파이썬 기본)	타입에 덧붙임 정보를 다는 도구	`Bearer`, `Db`, `CurrentUser` 별칭
`Depends`	"이 값은 이 함수로 만들어 넣어라"라는 표시	별칭 안
`HTTPBearer`, `HTTPAuthorizationCredentials`	`Authorization: Bearer …` 헤더를 읽는 도구와 그 결과	`bearer_scheme`, `credentials`
`Session`	DB 세션 타입 (SQLAlchemy)	`Db`
`AppError`, `ErrorCode`	01번의 오류 클래스와 코드	401
`decode_access_token`	05번의 토큰 검사	2단계
`get_db`	요청마다 DB 세션을 열고 닫는 함수	`Db`
`User`	users 테이블 모델	`db.get(User, …)`

함수 (이미 있던 별칭 포함)

```
bearer_scheme = HTTPBearer(auto_error=False)
Bearer = Annotated[HTTPAuthorizationCredentials | None, Depends(bearer_scheme)]
Db = Annotated[Session, Depends(get_db)]


def get_current_user(
    credentials: Bearer,
    db: Db,
) -> User:
    if credentials is None:
        raise AppError(ErrorCode.UNAUTHORIZED, 401, "Please sign in.")
    user_id = decode_access_token(credentials.credentials)
    user = db.get(User, user_id)
    if user is None or user.status != "ACTIVE":
        raise AppError(ErrorCode.UNAUTHORIZED, 401, "Please sign in again.")
    return user


CurrentUser = Annotated[User, Depends(get_current_user)]
```

문법 풀이

import 문법	뜻
`from 모듈 import 이름`	모듈 안의 `이름`만 가져와서 그대로 씁니다
`app.…`	이 프로젝트(`studionx/backend/app/`)의 파일입니다. `app.api.deps`는 `app/api/deps.py`입니다

식	뜻
`Depends(함수)`	FastAPI가 요청마다 그 함수를 먼저 실행해서 결과를 인자로 넣어 줍니다 ("의존성")
`Annotated[타입, Depends(…)]`	타입과 `Depends`를 한 이름으로 묶습니다. 그래서 `user: CurrentUser` 한 번만 쓰면 됩니다
`HTTPBearer(auto_error=False)`	헤더가 없거나 `Bearer`로 시작하지 않으면 **오류 대신 `None`**을 넣어 줍니다. 기본값(`True`)이라면 FastAPI가 그 자리에서 401 오류를 던집니다
`credentials.credentials`	`Bearer eyJ…`에서 `eyJ…` 부분입니다 (`credentials.scheme`은 `"Bearer"`입니다)
`db.get(User, 7)`	기본 키(`user_id`)가 7인 행을 찾습니다. 없으면 `None`입니다
`A is None or B`	`A`가 참이면 `B`를 보지 않습니다. 그래서 `None.status` 오류가 나지 않습니다

요청	결과
헤더 없음	401 "Please sign in."
`Authorization: Basic …`	401 "Please sign in." (`Bearer`가 아니라서 `None`이 됩니다)
토큰이 틀리거나 만료됨	401 "Please sign in again." (05번이 던집니다)
토큰은 맞는데 회원이 없음	401 "Please sign in again."
회원의 `status`가 `SUSPENDED` 또는 `DELETED`	401 "Please sign in again."
회원의 `status`가 `ACTIVE`	그 회원(`User`)을 돌려줍니다

4. 흐름

1. 헤더가 없으면 401 "Please sign in."으로 응답합니다.
2. 토큰을 검사해서 `user_id`를 얻습니다 (05). 실패하면 401로 응답합니다.
3. DB에서 그 회원을 찾습니다.
4. 회원이 없거나 `ACTIVE`가 아니면 401 "Please sign in again."으로 응답합니다.
5. 회원을 돌려줍니다.

5. 왜 이렇게

선택	이유
토큰만 믿지 않고 DB를 한 번 더 확인	Access Token은 30분 동안 유효합니다. 그사이 탈퇴(`DELETED`)하거나 정지(`SUSPENDED`)된 회원의 토큰을 막기 위해서입니다 (FR-108)
없는 회원, 정지, 탈퇴를 같은 401과 같은 문구로 처리	05번과 같은 이유입니다. 이유를 알려 주면 "이 번호의 회원은 탈퇴했다" 같은 정보가 새어 나갑니다
403이 아니라 401로 응답	프론트 `api.js`는 401 `UNAUTHORIZED`일 때만 재발급을 시도하고, 실패하면 로그인 화면으로 보냅니다 (JS 02). 정지되거나 탈퇴한 회원은 재발급도 거절되므로(P1-05) 결국 로그인 화면으로 갑니다

선택	이유
문구를 두 가지로 나눔	헤더가 없으면 처음부터 로그인하지 않은 것이므로 "Please sign in."이고, 토큰이 있는데 안 되면 "Please sign in again."입니다
`auto_error=False`로 두고 직접 401을 던짐	같은 `Bearer`를 07번 `get_current_user_optional()`도 씁니다. 07번은 헤더가 없을 때 오류가 아니라 `None`(손님)으로 처리해야 합니다. 그래서 헤더 읽기는 오류를 내지 않게 두고, 판단은 각 함수가 합니다
`status != "ACTIVE"`를 문자열로 비교	users 모델의 `status` 칸은 이 작업의 다음 순서(모델 컬럼)에서 만듭니다. 그때 `StrEnum`으로 만들면 `UserStatus.ACTIVE == "ACTIVE"`가 참이므로, 이 코드는 그대로 동작합니다

6. 시험 (`tests/test_deps.py` 새 파일, 7개)

users 모델의 컬럼(`status` 등)이 아직 없어서, 진짜 DB 대신 **가짜 세션(`FakeDb`)**을 넣었습니다. `FakeDb`는 `db.get(User, id)`만 흉내 내고, 받은 인자를 기록합니다. 시험용 앱에 `GET /whoami`(`user: CurrentUser` → `{"user_id": …}`)를 만들어 요청했습니다.

경우	결과
정상 토큰과 `ACTIVE` 회원 7	200 `{"user_id": 7}`이고, DB에 `(User, 7)`로 한 번 물어봅니다
헤더 없음	401 "Please sign in."이고, DB에 묻지 않습니다
`Authorization: Basic …`	401 "Please sign in."
틀린 토큰	401 "Please sign in again."이고, DB에 묻지 않습니다
토큰의 회원 99가 DB에 없음	401 "Please sign in again."
`SUSPENDED`, `DELETED` 회원 (2개)	401 "Please sign in again."

`app.dependency_overrides[get_db] = …`는 시험에서 `get_db` 대신 다른 함수를 쓰게 하는 FastAPI 기능입니다.

전체 시험 42개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 07 — `get_current_user_optional()`

항목	내용
파일	`studionx/backend/app/api/deps.py`
작업	P1-03 백엔드 공통: 로그인이 선택인 API (PRD 01 FR-112)
작성	Claude (AI), 2026-10-06, 사용자 검토
이전 · 다음	06 `get_current_user()` · 08 `User` 모델

1. 하는 일

로그인한 회원이면 그 회원을, 아니면 `None`(손님)을 라우터에 넘깁니다. **401 오류를 내지 않습니다.** 검사 내용은 06번과 같고, 실패했을 때의 처리만 다릅니다.

	06 `get_current_user()`	07 이 함수
로그인한 `ACTIVE` 회원	회원을 넘깁니다	회원을 넘깁니다
헤더 없음, 토큰 틀림, 회원 없음, 정지, 탈퇴	401 오류를 내고 라우터를 실행하지 않습니다	`None`을 넘기고, 라우터는 손님으로 실행됩니다
라우터에 적는 이름	`user: CurrentUser`	`user: OptionalUser`

2. 언제 쓰나

지금 쓰는 곳은 **문의 등록** `POST /api/v1/inquiries` 하나입니다 (home.html 아래쪽의 Contact 폼).

보내는 사람	이 함수의 결과	저장되는 문의 (`inquiries` 테이블)
로그인하지 않은 방문자	`None`	`user_id = NULL`이고, 폼에 적은 이메일로만 답장합니다
로그인한 7번 회원	7번 회원	`user_id = 7`이므로, 관리자가 어느 회원의 문의인지 알 수 있습니다
토큰이 만료된 회원	`None`	`user_id = NULL`이고, 문의는 그대로 접수됩니다

```
def create_inquiry(data: InquiryCreateRequest, request: Request, user: OptionalUser, db: Db):
    ...   # P1-12: user 가 있으면 inquiry.user_id = user.user_id
```

3. 코드

import (파일 맨 위)

```
from typing import Annotated

from fastapi import Depends
from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer
from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.core.security import decode_access_token
```

```
from app.db.session import get_db
from app.models import User
```

`deps.py` 맨 위의 import입니다 (06번과 같음). 이 함수는 **`AppError`**만 직접 씁니다. 나머지 검사는 06번 `get_current_user()`를 불러서 합니다.

함수

```
def get_current_user_optional(
    credentials: Bearer,
    db: Db,
) -> User | None:
    if credentials is None:
        return None
    try:
        return get_current_user(credentials, db)
    except AppError:
        return None


OptionalUser = Annotated[User | None, Depends(get_current_user_optional)]
```

문법 풀이

import 문법	뜻
`from 모듈 import 이름`	모듈 안의 `이름`만 가져와서 그대로 씁니다

식	뜻
`-> User | None`	회원 또는 `None`을 돌려준다는 표시입니다
`get_current_user(credentials, db)`	06번을 **보통 함수처럼 직접** 부릅니다. `Depends`를 거치지 않으므로, 이미 받은 `credentials`와 `db`를 그대로 넘깁니다
`try: … except AppError: return None`	06번이 던진 401(`AppError`)을 잡아서 `None`으로 바꿉니다. 05번의 토큰 오류도 `AppError`라서 함께 잡힙니다
`except AppError` (모든 오류를 잡지 않음)	DB 연결이 끊긴 오류 같은 것은 잡지 않습니다. 그런 오류는 04번이 500으로 처리합니다

4. 흐름

1. 헤더가 없으면 `None`을 돌려줍니다. DB에 묻지 않습니다.
2. 헤더가 있으면 06번과 같은 검사를 합니다.
3. 통과하면 회원을, 401이 나면 `None`을 돌려줍니다.

5. 왜 이렇게

선택	이유
실패해도 401을 내지 않음	문의는 손님도 보낼 수 있습니다. 만료된 토큰 때문에 문의가 거절되면 안 됩니다
검사를 06번에 맡김	검사 규칙(토큰, 회원 없음, 정지, 탈퇴)이 한 곳에만 있게 됩니다. 06번을 고치면 07번도 같이 바뀝니다

선택	이유
`AppError`만 잡음	DB 오류까지 `None`으로 바꾸면 서버 고장이 숨겨집니다. 고장은 04번이 500으로 알리고 Sentry에 기록합니다
정지되거나 탈퇴한 회원도 `None`	그 회원 이름으로 연결하지 않고 손님 문의로 받습니다

P1-12에서 알아 둘 점: Access Token은 메모리에만 있어서 home.html을 새로 열면 비어 있습니다 (JS 01). 로그인한 회원의 문의를 회원과 연결하려면, `home.js`가 보내기 전에 `isLoggedIn()`(JS 06)으로 토큰을 먼저 받아 두어야 합니다.

6. 시험 (`tests/test_deps.py`에 6개를 추가해서 모두 13개)

06번과 같은 가짜 세션(`FakeDb`)을 쓰고, 시험용 앱에 `GET /maybe(user: OptionalUser` → `{"user_id": 7 또는 null}`)를 만들었습니다.

경우	결과
정상 토큰과 `ACTIVE` 회원 7	200 `{"user_id": 7}`
헤더 없음	200 `{"user_id": null}`이고, DB에 묻지 않습니다
틀린 토큰	200 `{"user_id": null}`
토큰의 회원 99가 DB에 없음	200 `{"user_id": null}`
`DELETED` 회원	200 `{"user_id": null}`
`Authorization: Basic …`	200 `{"user_id": null}`

전체 시험 48개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 08 — `User` 모델 컬럼

항목	내용
파일	`studionx/backend/app/models/user.py`
작업	P1-03 백엔드 공통: ORM 모델 컬럼 (`docs/DB.md` §3.1, `studionx/db/01_phase1_account.sql`)
작성	Claude (AI), 2026-10-06, 사용자 검토
이전 · 다음	07 `get_current_user_optional()` · 09 `Inquiry` 모델

1. 하는 일

MySQL의 `users` 테이블을 **파이썬 클래스 `User`**로 옮겨 적습니다. 그러면 SQL 문 대신 파이썬 코드로 회원을 읽고 쓸 수 있습니다.

SQL	파이썬 (이 클래스가 있으면)
`SELECT * FROM users WHERE user_id = 7`	`db.get(User, 7)`
`user.status` 칸 읽기	`user.status`는 `UserStatus.ACTIVE`
`INSERT INTO users (email, …) VALUES (…)`	`db.add(User(email=…, …))`

테이블은 이 클래스가 만들지 않습니다. 테이블은 `studionx/db/01_phase1_account.sql`로 이미 만들어져 있고, 이 클래스는 그 모양을 그대로 따라 적은 것입니다. 지금까지는 `user_id` 한 칸만 있었고, 이번에 나머지 14칸을 채웠습니다.

2. 언제 쓰나

회원을 다루는 모든 백엔드 코드가 이 클래스를 씁니다.

쓰는 곳	읽거나 쓰는 칸
06 `get_current_user()`	`status`가 `ACTIVE`인지 확인합니다
P1-04 회원가입	`email`, `password_hash`, `nickname`, `privacy_agreed_at`, `privacy_version`을 넣습니다
P1-05 로그인	`email`로 찾고, `password_hash`를 비교하며, `last_login_at`을 기록합니다
P1-08 회원 정보 (userInfo.html)	`nickname`과 `profile_image_path`를 수정하고, 탈퇴할 때 `status = DELETED`와 `deleted_at`을 채웁니다
P1-16 대시보드 (myPage.html)	`storage_quota_bytes`와 `storage_used_bytes`로 저장 공간 막대를 그립니다
P1-21 관리자 화면	`role`이 `ADMIN`인 회원만 들어갑니다

3. 코드

import (파일 맨 위)

```
from datetime import datetime
from enum import StrEnum
```

```
from sqlalchemy import BigInteger, Enum, FetchedValue, String, text
from sqlalchemy.dialects.mysql import DATETIME
from sqlalchemy.orm import Mapped, mapped_column

from app.db.session import Base
```

`BigInteger`, `Mapped`, `mapped_column`, `Base`는 원래 있었고, 나머지를 이번에 추가했습니다. 모두 이 클래스에서 씁니다.

import	가져오는 것	쓰는 곳
`datetime` (파이썬 기본)	날짜와 시각 타입	`created_at` 등 시각 칸
`StrEnum` (파이썬 기본)	값이 문자열인 열거형	`UserRole`, `UserStatus`
`BigInteger`, `String`	SQL의 `BIGINT`, `VARCHAR(n)`	숫자 칸, 글자 칸
`Enum`	SQL의 `ENUM(…)`	`role`, `status`
`text`	SQL 문장을 그대로 넘깁니다	기본값 `CURRENT_TIMESTAMP(3)` 등
`FetchedValue`	"DB가 값을 바꾼다"는 표시	`updated_at`
`DATETIME` (MySQL 전용)	밀리초까지 저장하는 `DATETIME(3)`	시각 칸
`Mapped`, `mapped_column`	칸 선언	모든 칸
`Base`	모든 모델의 부모 클래스 (`app/db/session.py`)	`class User(Base)`

함수 (이 작업은 함수가 아니라 클래스와 칸 선언입니다)

```
class UserRole(StrEnum):
    USER = "USER"
    ADMIN = "ADMIN"


class UserStatus(StrEnum):
    ACTIVE = "ACTIVE"
    SUSPENDED = "SUSPENDED"
    DELETED = "DELETED"


class User(Base):
    __tablename__ = "users"

    user_id: Mapped[int] = mapped_column(BigInteger, primary_key=True, autoincrement=True)

    email: Mapped[str] = mapped_column(String(255), unique=True)
    password_hash: Mapped[str] = mapped_column(String(255))
    nickname: Mapped[str] = mapped_column(String(50), unique=True)
    profile_image_path: Mapped[str | None] = mapped_column(String(500))
    role: Mapped[UserRole] = mapped_column(Enum(UserRole), server_default=UserRole.USER)
    status: Mapped[UserStatus] = mapped_column(Enum(UserStatus), server_default=UserStatus.ACTIVE)
    last_login_at: Mapped[datetime | None] = mapped_column(DATETIME(fsp=3))
    privacy_agreed_at: Mapped[datetime] = mapped_column(DATETIME(fsp=3))
    privacy_version: Mapped[str] = mapped_column(String(20))
    storage_quota_bytes: Mapped[int] = mapped_column(BigInteger, server_default=text("5368709120"))
    storage_used_bytes: Mapped[int] = mapped_column(BigInteger, server_default=text("0"))
    created_at: Mapped[datetime] = mapped_column(
        DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)")
    )
```

```
    updated_at: Mapped[datetime] = mapped_column(
        DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)"), server_onupdate=FetchedValue()
    )
    deleted_at: Mapped[datetime | None] = mapped_column(DATETIME(fsp=3))
```

SQL과 한 줄씩 비교

SQL (01_phase1_account.sql)	파이썬
`email VARCHAR(255) NOT NULL` + `UNIQUE KEY uq_users_email`	`Mapped[str]`, `String(255)`, `unique=True`
`profile_image_path VARCHAR(500) NULL`	`Mapped[str | None]`
`role ENUM('USER','ADMIN') NOT NULL DEFAULT 'USER'`	`Enum(UserRole)`, `server_default=UserRole.USER`
`last_login_at DATETIME(3) NULL`	`Mapped[datetime | None]`, `DATETIME(fsp=3)`
`storage_quota_bytes BIGINT UNSIGNED NOT NULL DEFAULT 5368709120`	`BigInteger`, `server_default=text("5368709120")` (5GB)
`updated_at … DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3)`	`server_default=text(…)`, `server_onupdate=FetchedValue()`

문법 풀이

import 문법	뜻
`from 모듈 import 이름`	모듈 안의 `이름`만 가져와서 그대로 씁니다
`from sqlalchemy.dialects.mysql import …`	SQLAlchemy 안에 있는 **MySQL 전용** 타입입니다

식	뜻
`Mapped[str]`	이 칸의 파이썬 타입입니다. **`None`이 없으면 `NOT NULL`**이고, `Mapped[str | None]`이면 `NULL`을 허용합니다
`mapped_column(String(255))`	SQL 타입 `VARCHAR(255)`입니다
`unique=True`	같은 값을 두 번 넣을 수 없습니다 (SQL의 `UNIQUE KEY`)
`class UserStatus(StrEnum)`	정해진 값만 쓰게 하는 목록입니다. `UserStatus.ACTIVE == "ACTIVE"`가 참입니다
`server_default=…`	값을 넣지 않으면 **DB가** 채우는 기본값입니다. 파이썬은 넣지 않고 DB에 맡깁니다
`text("CURRENT_TIMESTAMP(3)")`	문자열 `'CURRENT_TIMESTAMP(3)'`가 아니라 SQL 식으로 넘깁니다
`server_onupdate=FetchedValue()`	"행을 고치면 DB가 이 칸을 바꾼다"는 표시입니다. 저장한 뒤 새 값을 다시 읽어 오게 합니다
`DATETIME(fsp=3)`	`DATETIME(3)`입니다. `fsp`는 초 아래 자릿수이며, 3은 밀리초입니다

4. 흐름 (회원 하나를 넣을 때)

1. `User(email=…, password_hash=…, nickname=…, privacy_agreed_at=…, privacy_version=…)`를 만듭니다. 다른 칸은 비워 둡니다.
2. `db.add(user)` → `db.flush()`로 DB에 `INSERT`합니다.
3. DB가 `user_id`, `role=USER`, `status=ACTIVE`, `storage_quota_bytes=5GB`, `created_at`을 채웁니다.
4. `db.refresh(user)`로 채워진 값을 다시 읽으면 `user.status`는 `UserStatus.ACTIVE`입니다.

5. 왜 이렇게

선택	이유
기본값은 `server_default`로 DB가 채움	기본값의 원본은 SQL 파일 하나입니다. 파이썬에도 따로 적으면 둘이 달라질 수 있습니다. 또 SQLAdmin이나 DB 도구로 직접 넣어도 같은 값이 들어갑니다
`role`과 `status`를 `StrEnum`으로	오타(`"ACTIVEE"`)를 막고, 편집기가 값 목록을 보여 줍니다. 06번의 `user.status != "ACTIVE"`도 그대로 동작합니다
시각 칸은 `DATETIME(3)` (시간대 없음)	DB에는 UTC로 저장하기로 했습니다 (`docs/DB.md`). 읽은 값에는 시간대 정보가 없으므로 UTC로 보고 씁니다
`BIGINT UNSIGNED`를 `BigInteger`로	테이블은 SQL 파일이 만들므로 부호 여부는 DB가 지킵니다. 파이썬에서 읽고 쓰는 데는 차이가 없고, 다른 모델과도 같습니다
관계(`relationship`)는 아직 넣지 않음	`user.projects`처럼 쓸 곳이 생길 때(P1-09 등) 넣습니다. 지금 넣으면 쓰지 않는 코드가 됩니다

6. 시험 (`tests/test_user_model.py` 새 파일, 4개)

실제 MySQL의 `users` 테이블과 비교했습니다. DB에 연결할 수 없으면 건너뜁니다. 넣은 행은 시험이 끝나면 되돌리므로(rollback) DB에 남지 않습니다 (시험 뒤 확인: 0행).

시험	확인한 것
칸이 DB와 같은지	칸 이름 15개, 순서, `NULL` 허용 여부가 실제 테이블과 같습니다
중복 금지 칸	`email`과 `nickname` 두 칸입니다
넣으면 DB가 기본값을 채우는지	5칸만 넣고 저장하면 `role`은 `USER`, `status`는 `ACTIVE`, 저장 공간은 5GB, 사용량은 0이 되고, `created_at`은 채워지며, `last_login_at`과 `deleted_at`은 비어 있습니다
06번 `get_current_user()`를 실제 DB로	`ACTIVE` 회원이면 그 회원을 돌려주고, `DELETED` 회원이면 401입니다

06번과 07번 시험은 가짜 DB 세션으로 했는데, 이번에 칸이 생겼으므로 실제 DB로도 확인했습니다.

전체 시험 52개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 09 — `Inquiry` 모델 컬럼

항목	내용
파일	`studionx/backend/app/models/inquiry.py`
작업	P1-03 백엔드 공통: ORM 모델 컬럼 (`docs/DB.md` §3.1, `studionx/db/01_phase1_account.sql`)
작성	Claude (AI), 2026-10-06, 사용자 검토
이전 · 다음	08 `User` 모델 · 10 `Project` 모델

1. 하는 일

MySQL의 `inquiries` 테이블(문의)을 파이썬 클래스 `Inquiry`로 옮겨 적습니다. 08번 `User`와 같은 방식이며, 이번에 새로 나온 것은 **다른 테이블을 가리키는 칸(FK)**과 **긴 글을 담는 칸(`TEXT`)**입니다.

새로 나온 것	칸	뜻
FK (외래 키)	`user_id` → `users.user_id`	이 문의를 보낸 회원입니다. 손님이면 비어 있습니다
`TEXT`	`content`, `answer`	길이 제한이 거의 없는 글입니다 (`VARCHAR`는 길이를 정해야 합니다)

2. 언제 쓰나

쓰는 곳	하는 일	칸
P1-12 문의 등록 (home.html의 Contact 폼)	문의를 저장합니다	`user_id`(07번의 결과), `email`, `title`, `content`, `privacy_agreed_at`
P1-21 관리자 화면 (SQLAdmin)	목록을 보고 답변합니다	`status`, `answer`, `answered_at`

문의 상태 (`status`)	뜻
`RECEIVED` (기본)	접수되었고, 아직 답변이 없습니다
`ANSWERED`	관리자가 답변했습니다
`CLOSED`	처리가 끝났습니다

3. 코드

import (파일 맨 위)

```
from datetime import datetime
from enum import StrEnum

from sqlalchemy import BigInteger, Enum, FetchedValue, ForeignKey, String, Text, text
from sqlalchemy.dialects.mysql import DATETIME
from sqlalchemy.orm import Mapped, mapped_column

from app.db.session import Base
```

`BigInteger`, `Mapped`, `mapped_column`, `Base`는 원래 있었고, 나머지를 이번에 추가했습니다. 08번과 같으며, **`ForeignKey`와 `Text`** 두 개가 새로 들어갔습니다.

import	가져오는 것	쓰는 곳
`ForeignKey`	다른 테이블의 칸을 가리키는 표시	`user_id`
`Text`	SQL의 `TEXT`	`content`, `answer`
나머지	08번의 표와 같습니다	

함수 (클래스와 칸 선언)

```
class InquiryStatus(StrEnum):
    RECEIVED = "RECEIVED"
    ANSWERED = "ANSWERED"
    CLOSED = "CLOSED"


class Inquiry(Base):
    __tablename__ = "inquiries"

    inquiry_id: Mapped[int] = mapped_column(BigInteger, primary_key=True, autoincrement=True)

    user_id: Mapped[int | None] = mapped_column(BigInteger, ForeignKey("users.user_id"))
    name: Mapped[str | None] = mapped_column(String(50))
    email: Mapped[str] = mapped_column(String(255))
    title: Mapped[str] = mapped_column(String(200))
    content: Mapped[str] = mapped_column(Text)
    privacy_agreed_at: Mapped[datetime] = mapped_column(DATETIME(fsp=3))
    status: Mapped[InquiryStatus] = mapped_column(
        Enum(InquiryStatus), server_default=InquiryStatus.RECEIVED
    )
    answer: Mapped[str | None] = mapped_column(Text)
    answered_at: Mapped[datetime | None] = mapped_column(DATETIME(fsp=3))
    created_at: Mapped[datetime] = mapped_column(
        DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)")
    )
    updated_at: Mapped[datetime] = mapped_column(
        DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)"), server_onupdate=FetchedValue()
    )
```

SQL과 비교 (08번에 없던 것만)

SQL (`01_phase1_account.sql`)	파이썬
`user_id BIGINT UNSIGNED NULL` + `FOREIGN KEY (user_id) REFERENCES users (user_id)`	`Mapped[int | None]`, `ForeignKey("users.user_id")`
`content TEXT NOT NULL`	`Mapped[str]`, `Text`
`KEY idx_inquiries_status (status)` (검색용 색인)	적지 않았습니다 (아래 §5 참고)

문법 풀이

import 문법	뜻
`from 모듈 import 이름`	모듈 안의 `이름`만 가져와서 그대로 씁니다

식	뜻
`ForeignKey("users.user_id")`	"이 칸의 값은 `users` 테이블의 `user_id` 중 하나"라는 표시입니다. `"테이블.칸"` 형식의 문자열로 적습니다
`Mapped[int | None]` + FK	비어 있어도 됩니다(손님 문의). 값이 있으면 실제 회원 번호여야 합니다
`Text`	길이를 적지 않습니다. MySQL의 `TEXT`는 약 64KB(한글 약 2만 자)입니다

4. 흐름 (손님 문의 하나를 넣을 때)

1. `Inquiry(email=…, title=…, content=…, privacy_agreed_at=…)`처럼 4칸만 넣어서 만듭니다.
2. `db.add` → `db.flush()`로 `INSERT`합니다.
3. DB가 `inquiry_id`, `status=RECEIVED`, `created_at`을 채웁니다. `user_id`는 비어 있습니다.
4. 로그인한 회원이라면 1단계에서 `user_id=7`을 함께 넣습니다. 없는 번호라면 DB가 거절합니다.

5. 왜 이렇게

선택	이유
`user_id`를 비울 수 있게 함	손님도 문의할 수 있기 때문입니다 (FR-112). 07번이 `None`을 넘기면 그대로 비워 둡니다
FK를 파이썬에도 적음	테이블은 SQL이 만들지만, 적어 두면 나중에 `relationship`(문의 → 회원)을 붙일 수 있고, 코드만 봐도 연결을 알 수 있습니다
색인(`idx_inquiries_status`)은 적지 않음	색인은 DB가 검색을 빠르게 하려고 가진 것이라서, 파이썬에서 읽고 쓰는 데는 필요하지 않습니다. 08번도 마찬가지입니다
`content` 최대 2,000자 검사는 여기서 하지 않음	칸(`TEXT`)은 2,000자보다 훨씬 큽니다. 글자 수 검사는 입력 스키마(P1-12, 02번의 422)가 합니다
`name` 칸도 적음	폼에서는 받지 않는 예비 칸이지만, 테이블에 있으므로 모델도 같게 맞춥니다
회원이 탈퇴해도 문의는 남김	FK에 `ON DELETE`가 없습니다. 탈퇴는 행을 지우지 않고 `status=DELETED`로 바꾸는 방식이므로(08), 문의의 `user_id`도 그대로 남습니다

6. 시험

`tests/test_inquiry_model.py` (새 파일, 5개): 실제 MySQL과 비교했습니다. 넣은 행은 시험이 끝나면 되돌립니다 (시험 뒤 확인: 0행).

시험	확인한 것
칸이 DB와 같은지	칸 12개의 이름, 순서, `NULL` 허용 여부, FK(`user_id` → `users.user_id`)가 실제 테이블과 같습니다
손님 문의	4칸만 넣으면 `user_id`는 비어 있고, `status`는 `RECEIVED`이며, `created_at`은 채워지고, 답변 칸은 비어 있습니다
회원 문의	회원을 만들고 그 번호를 넣으면 저장됩니다
없는 회원 번호 (999999999)	DB가 거절합니다 (`IntegrityError`)
2,000자 글	그대로 저장되고 다시 읽힙니다

`tests/conftest.py` 정리 (모든 모델 시험이 함께 씁니다)

추가한 것	하는 일
`db`	실제 MySQL 세션입니다. 연결할 수 없으면 시험을 건너뛰고, 끝나면 되돌립니다(rollback)
`assert_matches_db(모델)`	모델의 칸 이름, 순서, `NULL` 허용 여부, FK를 실제 테이블과 비교합니다

08번의 `tests/test_user_model.py`에 있던 같은 코드를 여기로 옮겼고, 08번 시험도 이것을 쓰도록 바꿨습니다. 남은 모델(10번부터)도 이것을 씁니다.

전체 시험 57개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 10 — `Project` 모델 컬럼

항목	내용
파일	`studionx/backend/app/models/project.py` (`Project` 클래스)
작업	P1-03 백엔드 공통: ORM 모델 컬럼 (`docs/DB.md` §3.1, `studionx/db/01_phase1_account.sql`)
작성	Claude (AI), 2026-10-06, 사용자 검토
이전 · 다음	09 `Inquiry` 모델 · 11 `ProjectMember` 모델

1. 하는 일

MySQL의 `projects` 테이블(시퀀서 프로젝트 한 개)을 파이썬 클래스 `Project`로 옮겨 적습니다. 08번, 09번과 같은 방식이며, 이번에 새로 나온 타입은 세 가지입니다.

새로 나온 타입	칸	파이썬에서 읽으면
`DECIMAL(6,2)` (정확한 소수)	`bpm`	`Decimal("120.00")`
`TINYINT UNSIGNED` (0~255)	`time_sig_num`, `time_sig_den` (박자 4/4)	`4`
`JSON`	`master_settings` (마스터 채널 설정)	사전 `{"gain_db": -1.5}`

같은 파일의 `ProjectMember`는 11번에서 다루고, 관계(`project.members`, `project.tracks`)는 `ProjectMember`와 `Track` 모델을 채운 뒤에 붙입니다.

2. 언제 쓰나

쓰는 곳	화면	칸
P1-13 프로젝트 목록, 만들기, 이름 바꾸기, 삭제	myProject.html	`owner_id`, `title`, `description`, `updated_at`, `deleted_at`
P1-16 대시보드의 최근 프로젝트	myPage.html	`title`, `updated_at`, `thumbnail_path`
Phase 2 시퀀서	studioNx.html 위쪽 막대 (BPM, 박자, 키)	`bpm`, `time_sig_*`, `music_key`, `scale_mode`, `sample_rate`, `master_settings`, `revision`

3. 코드

import (파일 맨 위)

```
from datetime import datetime
from decimal import Decimal
from typing import Any

from sqlalchemy import JSON, BigInteger, FetchedValue, ForeignKey, Integer, Numeric, String, text
from sqlalchemy.dialects.mysql import DATETIME, TINYINT
from sqlalchemy.orm import Mapped, mapped_column

from app.db.session import Base
```

`BigInteger`, `ForeignKey`, `Mapped`, `mapped_column`, `Base`는 원래 있었고, 나머지를 이번에 추가했습니다.
`ProjectMember`(11)도 같은 import를 씁니다.

import	가져오는 것	쓰는 곳
`Decimal` (파이썬 기본)	정확한 소수	`bpm`
`Any` (파이썬 기본)	"아무 타입"이라는 표시	`dict[str, Any]`: JSON 안의 값
`Numeric(6, 2)`	SQL의 `DECIMAL(6,2)`	`bpm`
`TINYINT` (MySQL 전용)	1바이트 정수	박자
`Integer`	SQL의 `INT`	`sample_rate`, `revision`
`JSON`	SQL의 `JSON`	`master_settings`
나머지	08번의 표와 같습니다	

함수 (클래스와 칸 선언)

```
class Project(Base):
    __tablename__ = "projects"

    project_id: Mapped[int] = mapped_column(BigInteger, primary_key=True, autoincrement=True)

    owner_id: Mapped[int] = mapped_column(BigInteger, ForeignKey("users.user_id"))
    title: Mapped[str] = mapped_column(String(100))
    description: Mapped[str | None] = mapped_column(String(500))
    bpm: Mapped[Decimal] = mapped_column(Numeric(6, 2), server_default=text("120.00"))
    time_sig_num: Mapped[int] = mapped_column(TINYINT(unsigned=True), server_default=text("4"))
    time_sig_den: Mapped[int] = mapped_column(TINYINT(unsigned=True), server_default=text("4"))
    music_key: Mapped[str | None] = mapped_column(String(10))
    scale_mode: Mapped[str | None] = mapped_column(String(20))
    sample_rate: Mapped[int] = mapped_column(Integer, server_default=text("48000"))
    master_settings: Mapped[dict[str, Any] | None] = mapped_column(JSON)
    revision: Mapped[int] = mapped_column(Integer, server_default=text("0"))
    thumbnail_path: Mapped[str | None] = mapped_column(String(500))
    # FK → project_templates (Phase 2). 그 모델이 생기면 ForeignKey 를 붙인다.
    template_id: Mapped[int | None] = mapped_column(BigInteger)
    created_at: Mapped[datetime] = mapped_column(
        DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)")
    )
    updated_at: Mapped[datetime] = mapped_column(
        DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)"), server_onupdate=FetchedValue()
    )
    deleted_at: Mapped[datetime | None] = mapped_column(DATETIME(fsp=3))
```

SQL과 비교 (앞에 없던 것만)

SQL (`01_phase1_account.sql`)	파이썬
`bpm DECIMAL(6,2) NOT NULL DEFAULT 120.00`	`Mapped[Decimal]`, `Numeric(6, 2)`, `server_default=text("120.00")`
`time_sig_num TINYINT UNSIGNED NOT NULL DEFAULT 4`	`Mapped[int]`, `TINYINT(unsigned=True)`
`master_settings JSON NULL`	`Mapped[dict[str, Any] | None]`, `JSON`
`template_id BIGINT UNSIGNED NULL` + (Phase 2에서 `ALTER`로 FK 추가)	`Mapped[int | None]`이고, **FK는 적지 않았습니다** (아래 §5 참고)

문법 풀이

import 문법	뜻
`from 모듈 import 이름`	모듈 안의 `이름`만 가져와서 그대로 씁니다
`from sqlalchemy.dialects.mysql import …`	SQLAlchemy 안에 있는 MySQL 전용 타입입니다

식	뜻
`Numeric(6, 2)`	전체 6자리 중 소수점 아래가 2자리이므로, 최대 `9999.99`입니다
`Decimal("98.50")`	소수를 정확하게 저장합니다. `float`의 `0.1 + 0.2 = 0.30000000000000004` 같은 오차가 없습니다
`TINYINT(unsigned=True)`	0~255의 정수입니다
`dict[str, Any]`	열쇠는 문자열이고 값은 아무 타입인 사전입니다. JSON 객체와 같은 모양입니다
`mapped_column(JSON)`	저장할 때는 사전을 JSON 문자열로, 읽을 때는 JSON 문자열을 사전으로 자동으로 바꿉니다

4. 흐름 (새 프로젝트 하나를 넣을 때)

1. `Project(owner_id=7, title="My Song")`처럼 2칸만 넣어서 만듭니다.
2. `db.add` → `db.flush()`로 `INSERT`합니다.
3. DB가 `project_id`, `bpm=120.00`, 박자 `4/4`, `sample_rate=48000`, `revision=0`, `created_at`을 채웁니다.
4. P1-13에서는 같은 저장 안에서 OWNER 멤버(11)와 MASTER 트랙(12)도 함께 넣습니다 (FR-122).

5. 왜 이렇게

선택	이유
`bpm`은 `Decimal`	화면에서 `98.50`을 넣으면 `98.50`으로 돌아와야 합니다. 입력 스키마(`schemas/project.py`)도 `Decimal`이라서 그대로 넘길 수 있습니다
`template_id`에 FK를 적지 않음	가리킬 `project_templates` 모델이 아직 없습니다(Phase 2). 없는 테이블을 가리키면 SQLAlchemy가 저장할 때 오류를 냅니다. 실제 FK는 DB에 있으므로 잘못된 번호는 DB가 막습니다
`revision`은 보통 칸으로 둠	저장할 때마다 1씩 올려서 동시 저장 충돌을 찾는 데 씁니다. 어떻게 올릴지는 저장 기능(Phase 2)에서 정합니다
관계는 다음에 붙임	`project.members`와 `project.tracks`는 `ProjectMember`(11)와 `Track`(12)의 칸이 채워져야 붙일 수 있습니다
색인(`idx_projects_owner`, `idx_projects_updated`)은 적지 않음	08번, 09번과 같은 이유입니다. DB의 검색용입니다

6. 시험

`tests/test_project_model.py` (새 파일, 4개): 실제 MySQL로 확인했습니다. 넣은 행은 시험이 끝나면 되돌립니다 (시험 뒤 확인: 0행).

시험	확인한 것
칸이 DB와 같은지	칸 17개의 이름, 순서, `NULL` 허용 여부, FK(`owner_id` → `users`)가 같습니다
2칸만 넣기	`bpm`은 `120.00`, 박자는 `4/4`, `48000`, `revision 0`이고, `master_settings`는 비어 있으며, `created_at`이 채워집니다
값을 넣고 DB에서 다시 읽기	`bpm 98.50`, 박자 `3`, JSON `{"gain_db": -1.5, "limiter": {"on": true}}`가 그대로 돌아옵니다
없는 회원 번호를 `owner_id`로 넣음	DB가 거절합니다 (`IntegrityError`)

`tests/conftest.py`의 `assert_matches_db` 보완

바꾼 것	이유
FK의 `ON DELETE`도 비교	11번 `ProjectMember`의 `ON DELETE CASCADE`가 DB와 같은지 보기 위해서입니다
아직 모델이 없는 테이블을 가리키는 FK는 비교에서 뺌	`template_id → project_templates` (Phase 2)

전체 시험 61개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 11 — `ProjectMember` 모델 컬럼

항목	내용
파일	`studionx/backend/app/models/project.py` (`ProjectMember` 클래스)
작업	P1-03 백엔드 공통: ORM 모델 컬럼 (`docs/DB.md` §3.1, `studionx/db/01_phase1_account.sql`)
작성	Claude (AI), 2026-10-06, 사용자 검토
이전 · 다음	10 `Project` 모델 · 12 `Track` 모델

1. 하는 일

MySQL의 `project_members` 테이블(어느 회원이 어느 프로젝트에 어떤 권한으로 참여하는지)을 파이썬 클래스 `ProjectMember`로 옮겨 적습니다. 기본 키와 FK 2개는 뼈대에 이미 있었고, 이번에 나머지 3칸과 권한 목록 `MemberRole`을 채웠습니다.

project_id	user_id	role	뜻
12	7	`OWNER`	7번 회원이 12번 프로젝트를 만들었습니다
12	9	`EDITOR`	9번 회원은 12번 프로젝트를 고칠 수 있습니다 (Phase 6 협업)

`role`	뜻
`OWNER`	만든 사람입니다. 삭제와 초대를 할 수 있습니다
`EDITOR` (DB 기본값)	편집할 수 있습니다
`VIEWER`	보기만 할 수 있습니다

2. 언제 쓰나

쓰는 곳	하는 일
P1-13 프로젝트 만들기 (myProject.html의 **New project**)	프로젝트와 함께 `OWNER` 행 하나를 넣습니다 (FR-122)
P1-13, P1-16 목록	"내가 멤버인 프로젝트"를 찾을 때 이 테이블을 거칩니다
Phase 6 협업	초대받은 회원을 `EDITOR`나 `VIEWER`로 추가하고, 톡방에서 읽은 위치 (`last_read_message_id`)를 기록합니다

Phase 1에서는 **프로젝트마다 OWNER 한 행만** 씁니다.

3. 코드

import (파일 맨 위)

```
from datetime import datetime
from decimal import Decimal
from enum import StrEnum
from typing import Any
```

```
from sqlalchemy import (
    JSON,
    BigInteger,
    Enum,
    FetchedValue,
    ForeignKey,
    Integer,
    Numeric,
    String,
    text,
)
from sqlalchemy.dialects.mysql import DATETIME, TINYINT
from sqlalchemy.orm import Mapped, mapped_column

from app.db.session import Base
```

`project.py` 맨 위의 import입니다 (`Project`와 함께 씁니다). 이번에 **`StrEnum`과 `Enum`**을 추가했습니다. 줄이 길어져서 `ruff`가 괄호로 묶고 한 줄에 하나씩 적었습니다. `ProjectMember`는 이 중 `StrEnum`, `BigInteger`, `Enum`, `ForeignKey`, `text`, `DATETIME`, `Mapped`, `mapped_column`, `Base`를 씁니다.

함수 (클래스와 칸 선언)

```
class MemberRole(StrEnum):
    OWNER = "OWNER"
    EDITOR = "EDITOR"
    VIEWER = "VIEWER"


class ProjectMember(Base):
    __tablename__ = "project_members"

    # 복합 기본 키 (project_id, user_id)
    project_id: Mapped[int] = mapped_column(
        BigInteger, ForeignKey("projects.project_id", ondelete="CASCADE"), primary_key=True
    )
    user_id: Mapped[int] = mapped_column(BigInteger, ForeignKey("users.user_id"), primary_key=True)

    role: Mapped[MemberRole] = mapped_column(Enum(MemberRole), server_default=MemberRole.EDITOR)
    last_read_message_id: Mapped[int | None] = mapped_column(BigInteger)  # Phase 6 톡방
    joined_at: Mapped[datetime] = mapped_column(
        DATETIME(fsp=3), server_default=text("CURRENT_TIMESTAMP(3)")
    )
```

SQL과 비교

SQL (`01_phase1_account.sql`)	파이썬
`PRIMARY KEY (project_id, user_id)`	두 칸 모두 `primary_key=True`
`FOREIGN KEY (project_id) REFERENCES projects (project_id) ON DELETE CASCADE`	`ForeignKey("projects.project_id", ondelete="CASCADE")`
`role ENUM('OWNER','EDITOR','VIEWER') NOT NULL DEFAULT 'EDITOR'`	`Enum(MemberRole)`, `server_default=MemberRole.EDITOR`
`last_read_message_id BIGINT UNSIGNED NULL`	`Mapped[int | None]` (FK 없음: DB에도 없습니다)

SQL (01_phase1_account.sql)	파이썬
`joined_at DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3)`	`server_default=text(…)`

문법 풀이

import 문법	뜻
`from 모듈 import (A, B, …)`	가져올 이름이 많을 때 괄호로 묶어 여러 줄에 적습니다. 한 줄로 적은 것과 같습니다

식	뜻
복합 기본 키	두 칸을 **합쳐서** 한 행을 가리킵니다. `(12, 7)`은 한 번만 있을 수 있고, `(12, 9)`와 `(13, 7)`은 따로 있을 수 있습니다
`db.get(ProjectMember, (12, 7))`	복합 기본 키로 찾을 때는 값을 튜플로 넘깁니다
`ondelete="CASCADE"`	가리키는 프로젝트 행이 지워지면 **DB가** 이 행도 함께 지웁니다

4. 흐름 (프로젝트를 만들 때, P1-13)

1. `Project(owner_id=7, title=…)`를 넣고 `flush`해서 `project_id=12`를 받습니다.
2. `ProjectMember(project_id=12, user_id=7, role=MemberRole.OWNER)`를 넣습니다.
3. DB가 `joined_at`을 채웁니다.
4. 한 번에 `commit`합니다. 중간에 실패하면 둘 다 들어가지 않습니다 (FR-122).

5. 왜 이렇게

선택	이유
기본 키를 `(project_id, user_id)`로	같은 회원이 같은 프로젝트에 두 번 들어갈 수 없습니다. 그래서 따로 번호 칸이 필요하지 않습니다
`project_id`에 `ON DELETE CASCADE`	프로젝트 행이 정말 지워지면 멤버 행만 남을 이유가 없습니다. 보통 삭제는 `deleted_at`을 기록하는 소프트 삭제라서 이런 경우는 드뭅니다 (관리자가 DB에서 직접 지울 때 등)
`user_id`에는 `CASCADE`가 없음	회원은 지우지 않고 `status=DELETED`로 바꿉니다 (08). 회원 행을 실수로 지우려 하면 DB가 막습니다
`role` 기본값이 `EDITOR`	Phase 6에서 초대받는 사람은 보통 편집자이기 때문입니다. 만든 사람은 코드에서 반드시 `OWNER`를 적습니다
`last_read_message_id`에 FK가 없음	DB에도 없습니다. 가리킬 `chat_messages`는 Phase 6에서 만들고, 메시지가 지워져도 읽은 위치 숫자는 남아도 됩니다

6. 시험

`tests/test_project_member_model.py` **(새 파일, 5개)**: 실제 MySQL로 확인했습니다. 넣은 행은 시험이 끝나면 되돌립니다 (시험 뒤 확인: 0행).

시험	확인한 것
칸이 DB와 같은지	칸 5개, `NULL` 허용 여부, FK 2개(`project_id`는 `ON DELETE CASCADE`까지)가 같습니다

시험	확인한 것
기본 키	`["project_id", "user_id"]`
OWNER와 기본값	`role=OWNER`를 적으면 `OWNER`가 되고, 적지 않으면 DB가 `EDITOR`로 채웁니다. `joined_at`은 채워지고, `last_read_message_id`는 비어 있습니다
같은 회원을 두 번 넣음	DB가 거절합니다 (`IntegrityError`)
프로젝트 행을 지우면	멤버 수가 1에서 0이 됩니다 (DB의 `CASCADE`)

전체 시험 66개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 12 — `Track` 모델 컬럼 (Phase 1 칸만)

항목	내용
파일	`studionx/backend/app/models/track.py`
작업	P1-03 백엔드 공통: ORM 모델 컬럼 (`docs/DB.md` §3.2, `studionx/db/02_phase2_sequencer.sql`)
작성	Claude (AI), 2026-10-06, 사용자 검토
이전 · 다음	11 `ProjectMember` 모델 · 13 관계

1. 하는 일

MySQL의 `tracks` 테이블(시퀀서의 트랙 한 줄)을 파이썬 클래스 `Track`으로 옮겨 적습니다. 테이블에는 칸이 19개 있지만, **Phase 1에 필요한 3칸만** 채웠습니다. 나머지 14칸(색, 볼륨, 음소거 등)은 시퀀서를 만드는 Phase 2에서 채웁니다.

지금 모델에 있는 칸	뜻
`track_id`, `project_id` (뼈대에 있었음)	트랙 번호와 트랙이 속한 프로젝트입니다
`name`	트랙 이름입니다. 예: `Master`
`track_type`	트랙 종류입니다. `AUDIO`, `MIDI`, `MASTER`, `GROUP`, `RETURN` 중 하나입니다
`order_index`	화면에서의 순서입니다 (0부터)

2. 언제 쓰나

Phase 1에서는 **프로젝트를 만들 때 MASTER 트랙 1개를 넣는 데만** 씁니다 (P1-13, FR-122). studioNx.html을 처음 열면 맨 아래에 보이는 Master 트랙이 바로 이 행입니다.

```
# P1-13 project_service.create_project() 안 (예정)
db.add(Track(project_id=project.project_id, name="Master", track_type=TrackType.MASTER, order_index=0))
```

프로젝트를 만들 때 함께 넣는 것	모델
프로젝트 행	10 `Project`
OWNER 멤버 행	11 `ProjectMember`
MASTER 트랙 행	**12 `Track`**

3. 코드

import (파일 맨 위)

```
from enum import StrEnum

from sqlalchemy import BigInteger, Enum, ForeignKey, Integer, String, text
from sqlalchemy.orm import Mapped, mapped_column
```

```
from app.db.session import Base
```

`BigInteger`, `ForeignKey`, `Mapped`, `mapped_column`, `Base`는 원래 있었고, **`StrEnum`, `Enum`, `Integer`, `String`, `text`**를 이번에 추가했습니다. 모두 이 클래스에서 씁니다. 시각 칸(`created_at` 등)을 아직 넣지 않았기 때문에 `DATETIME`은 없습니다.

함수 (클래스와 칸 선언)

```
class TrackType(StrEnum):
    AUDIO = "AUDIO"
    MIDI = "MIDI"
    MASTER = "MASTER"
    GROUP = "GROUP"
    RETURN = "RETURN"


class Track(Base):
    __tablename__ = "tracks"

    track_id: Mapped[int] = mapped_column(BigInteger, primary_key=True, autoincrement=True)
    project_id: Mapped[int] = mapped_column(
        BigInteger, ForeignKey("projects.project_id", ondelete="CASCADE")
    )

    # Phase 1 칸 (MASTER 트랙 생성용). 나머지 칸은 DB 기본값 · NULL 로 채워진다
    name: Mapped[str] = mapped_column(String(100))
    track_type: Mapped[TrackType] = mapped_column(Enum(TrackType))
    order_index: Mapped[int] = mapped_column(Integer, server_default=text("0"))

    # TODO(Phase 2): color, volume_db, pan, is_muted, is_solo, … (뼈대의 주석 그대로)
```

SQL과 비교

SQL (`02_phase2_sequencer.sql`)	파이썬
`name VARCHAR(100) NOT NULL`	`Mapped[str]`, `String(100)`
`track_type ENUM('AUDIO',…,'RETURN') NOT NULL` (**기본값 없음**)	`Enum(TrackType)`이고, `server_default`가 없습니다
`order_index INT UNSIGNED NOT NULL DEFAULT 0`	`Integer`, `server_default=text("0")`
`color CHAR(7) NOT NULL DEFAULT '#7C5CFF'` 외 13칸	적지 않았습니다 (Phase 2). 행을 넣을 때 DB가 기본값이나 `NULL`로 채웁니다

문법 풀이

import 문법	뜻
`from 모듈 import 이름`	모듈 안의 `이름`만 가져와서 그대로 씁니다

식	뜻
모델에 없는 칸	SQLAlchemy는 모델에 적힌 칸만 `INSERT`하고 `SELECT`합니다. 나머지 칸은 DB가 기본값으로 채우고, 파이썬에서는 `track.color`처럼 읽을 수 없습니다

식	뜻
`track_type`에 `server_default`가 없음	반드시 넣어야 하는 칸입니다. 빠뜨리면 DB가 거절합니다

4. 흐름 (MASTER 트랙을 넣을 때)

1. `Track(project_id=12, name="Master", track_type=TrackType.MASTER)`를 만듭니다.
2. `db.add` → `db.flush()`로 `INSERT INTO tracks (project_id, name, track_type) …`를 실행합니다.
3. DB가 `track_id`, `order_index=0`과 모델에 없는 칸(`color='#7C5CFF'`, `volume_db=0.00`, `is_muted=0`, `routing_mode='CHAIN'` …)을 채웁니다.

5. 왜 이렇게

선택	이유
Phase 1 칸만 채움	뼈대의 TODO가 Phase 1 칸만 정해 두었습니다. 나머지 칸은 시퀀서(Phase 2)에서 의미가 정해지므로, 지금 적으면 쓰지 않는 코드가 됩니다. 모두 DB 기본값이나 `NULL`이 있어서 행을 넣을 때 문제가 없습니다
`track_type`에 기본값을 두지 않음	DB에도 기본값이 없습니다. 트랙 종류는 만드는 쪽이 반드시 정해야 합니다
FK는 `project_id` 하나만 적음	DB에는 FK가 5개 있지만, 나머지 4개(`parent_track_id`, `frozen_sample_id`, `assignee_id`, `locked_by`)는 Phase 2 칸에 붙어 있습니다. 그 칸을 채울 때 함께 적습니다
색인(`idx_tracks_project_order`)은 적지 않음	08~11번과 같은 이유입니다

6. 시험

`tests/test_track_model.py` (새 파일, 4개): 실제 MySQL로 확인했습니다. 넣은 행은 시험이 끝나면 되돌립니다 (시험 뒤 확인: 0행).

시험	확인한 것
칸이 DB와 같은지 (모델의 5칸만)	이름, 순서, `NULL` 허용 여부, `project_id` FK의 `ON DELETE CASCADE`가 같습니다
MASTER 트랙	`order_index`는 0입니다 (DB 기본값). 모델에 없는 칸도 SQL로 읽어 보니 `#7C5CFF`, `0.00`, `0`, `CHAIN`으로 채워져 있었습니다
`track_type`을 빠뜨리면	DB가 거절합니다 (`IntegrityError`)
프로젝트 행을 지우면	트랙 수가 1에서 0이 됩니다 (DB의 `CASCADE`)

`tests/conftest.py`의 `assert_matches_db` 보완: `partial=True`를 주면 모델에 있는 칸과 그 칸에 붙은 FK만 비교합니다. 다른 모델은 그대로 전체를 비교합니다.

전체 시험 70개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 13 — 모델 관계 (`project.members` · `project.tracks`)

항목	내용
파일	`studionx/backend/app/models/project.py`, `studionx/backend/app/models/track.py`
작업	P1-03 백엔드 공통: ORM 모델 관계 (P1-03의 마지막 작업)
작성	Claude (AI), 2026-10-06, 사용자 검토
이전 · 다음	12 `Track` 모델 · 14 `check_password_rules()` (P1-04)

1. 하는 일

10~12번의 FK(`project_id`)를 **파이썬 속성**으로도 오갈 수 있게 합니다. FK는 DB에서 숫자로 연결한 것이고, 관계(`relationship`)는 그 연결을 파이썬 객체로 따라갈 수 있게 해 줍니다.

관계가 없을 때	관계가 있을 때
`db.scalars(select(Track).where(Track.project_id == 12).order_by(Track.order_index))`	`project.tracks`
`db.get(Project, member.project_id)`	`member.project`
프로젝트를 `flush`해서 번호를 받은 뒤 `ProjectMember(project_id=12, …)`	`project.members.append(ProjectMember(…))`만 하면, 번호는 저장할 때 자동으로 채워집니다

속성	방향	값
`project.members`	프로젝트 → 멤버	`[ProjectMember, …]`
`project.tracks`	프로젝트 → 트랙	`[Track, …]` (`order_index` 순서)
`member.project`, `track.project`	반대 방향	`Project`

2. 언제 쓰나

쓰는 곳	쓰는 방법
P1-13 프로젝트 만들기 (FR-122)	프로젝트에 OWNER 멤버와 MASTER 트랙을 붙이고 `db.add(project)`를 한 번 하면 셋이 함께 저장됩니다
Phase 2 시퀀서	studioNx.html을 열 때 `project.tracks`로 트랙 목록을 화면 순서대로 읽습니다
Phase 6 협업	`project.members`로 멤버 목록과 권한을 확인합니다

```
# P1-13 create_project() 를 관계로 쓰면
project = Project(owner_id=user.user_id, title="My Song")
project.members.append(ProjectMember(user_id=user.user_id, role=MemberRole.OWNER))
project.tracks.append(Track(name="Master", track_type=TrackType.MASTER, order_index=0))
db.add(project)
db.commit()   # 하나라도 실패하면 셋 다 저장되지 않음
```

3. 코드

import (`project.py` 맨 위, 바뀐 줄만)

```
from typing import TYPE_CHECKING, Any
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.db.session import Base

if TYPE_CHECKING:  # 타입 표시용 (실행 중에는 불러오지 않음 — 서로 불러오는 순환을 피함)
    from app.models.track import Track
```

import (`track.py` 맨 위, 바뀐 줄만)

```
from typing import TYPE_CHECKING
from sqlalchemy.orm import Mapped, mapped_column, relationship

if TYPE_CHECKING:  # 타입 표시용 (실행 중에는 불러오지 않음)
    from app.models.project import Project
```

두 파일 모두 **`relationship`과 `TYPE_CHECKING`**을 추가했습니다. `project.py`는 `Track`을, `track.py`는 `Project`를 타입 표시에만 씁니다.

함수 (`project.py`의 `Project` 끝과 `ProjectMember` 끝)

```
class Project(Base):
    ...
    # 관계 — 행을 지우는 일은 DB 의 ON DELETE CASCADE 에 맡김 (passive_deletes)
    members: Mapped[list["ProjectMember"]] = relationship(
        back_populates="project", cascade="all, delete-orphan", passive_deletes=True
    )
    tracks: Mapped[list["Track"]] = relationship(
        back_populates="project",
        order_by="Track.order_index",
        cascade="all, delete-orphan",
        passive_deletes=True,
    )


class ProjectMember(Base):
    ...
    project: Mapped[Project] = relationship(back_populates="members")
```

(`track.py`의 `Track` 끝)

```
    project: Mapped["Project"] = relationship(back_populates="tracks")
```

문법 풀이

import 문법	뜻
`from typing import TYPE_CHECKING`	편집기나 타입 검사기가 읽을 때만 `True`이고, 실제로 실행할 때는 `False`인 값입니다

import 문법	뜻
`if TYPE_CHECKING: from … import Track`	타입 표시에만 쓰는 import입니다. `project.py`와 `track.py`가 실행 중에 서로를 불러오면 "아직 다 읽지 않은 파일" 오류(순환 import)가 날 수 있어서 이렇게 피합니다

식	뜻
`relationship()`	FK를 따라 연결된 객체를 속성으로 줍니다. 어느 FK를 쓸지는 SQLAlchemy가 칸을 보고 찾습니다
`Mapped[list["ProjectMember"]]`	여러 개(목록)라는 뜻입니다. `"…"`처럼 문자열로 적으면 아직 정의되지 않은 클래스도 가리킬 수 있습니다
`Mapped[Project]`	하나라는 뜻입니다
`back_populates="project"`	반대쪽 속성의 이름입니다. 한쪽을 바꾸면 다른 쪽도 맞춰집니다 (`member.project` ↔ `project.members`)
`order_by="Track.order_index"`	`project.tracks`를 읽을 때 이 순서로 정렬합니다
`cascade="all, delete-orphan"`	프로젝트를 `add`하거나 `delete`하면 자식도 함께 처리합니다. 목록에서 뺀 자식(고아)은 행도 지웁니다
`passive_deletes=True`	프로젝트를 지울 때 파이썬이 자식 행을 하나씩 지우지 않고, DB의 `ON DELETE CASCADE`에 맡깁니다

4. 흐름 (`db.add(project)` → `flush`)

1. `project.members`와 `project.tracks`에 붙은 객체도 함께 저장 대상이 됩니다 (`cascade`).
2. `INSERT INTO projects`를 실행해서 `project_id=12`를 받습니다.
3. 자식의 `project_id` 칸에 12를 자동으로 넣고 `INSERT INTO project_members`와 `INSERT INTO tracks`를 실행합니다.
4. 하나라도 실패하면 `rollback`으로 셋 다 없던 일이 됩니다.

5. 왜 이렇게

선택	이유
`members`는 뼈대 TODO대로 (`cascade="all, delete-orphan"`)	멤버는 프로젝트 없이 존재할 수 없습니다
`tracks`에도 `cascade`를 추가 (뼈대 TODO에는 `order_by`만 있었음)	이것이 없으면 트랙을 읽은 상태로 프로젝트를 지울 때, SQLAlchemy가 트랙의 `project_id`를 비우려다가 `NOT NULL` 오류를 냅니다. 트랙도 프로젝트 없이 존재할 수 없습니다
`passive_deletes=True`	DB에 이미 `ON DELETE CASCADE`가 있습니다(11, 12). 파이썬이 자식을 모두 읽어서 하나씩 지우면 느리고, 같은 일을 두 번 하게 됩니다
보통의 삭제는 소프트 삭제	P1-13의 삭제는 `deleted_at`을 기록하는 방식이라서 위의 삭제 동작은 쓰이지 않습니다. 관리자가 정말로 지우는 경우를 위한 것입니다
`User.projects`는 아직 넣지 않음	뼈대에 "필요해지면"이라고 되어 있습니다. 목록은 `ProjectMember`로 JOIN해서 찾습니다 (P1-13 TODO)
`TYPE_CHECKING`으로 import	두 파일이 서로를 가리키므로, 실행 중의 순환 import를 피하면서 편집기 자동 완성은 살립니다

6. 시험 (`tests/test_model_relationships.py` 새 파일, 6개)

실제 MySQL로 확인했습니다. 넣은 행은 시험이 끝나면 되돌립니다 (시험 뒤 확인: 0행).

시험	확인한 것
프로젝트만 `add`	멤버와 트랙도 저장되고, 자식의 `project_id`가 자동으로 채워집니다
반대 방향	`member.project is project`, `track.project is project`
트랙 순서	`order_index`를 2, 1 순서로 붙여도 다시 읽으면 `Master`, `Bass`, `Drums` 순서입니다
멤버를 목록에서 빼면	행이 지워집니다 (`delete-orphan`)
자식을 읽은 상태로 프로젝트를 지우면	오류 없이 멤버와 트랙이 0개가 됩니다
트랙 하나가 실패하면 (`track_type` 빠짐)	`IntegrityError`가 나고, `rollback`한 뒤에는 프로젝트도 없습니다 (FR-122)

전체 시험 76개가 통과했고, `ruff` 검사도 통과했습니다. 앱을 시작할 때의 관계 설정(`configure_mappers()`)도 오류가 없었습니다.